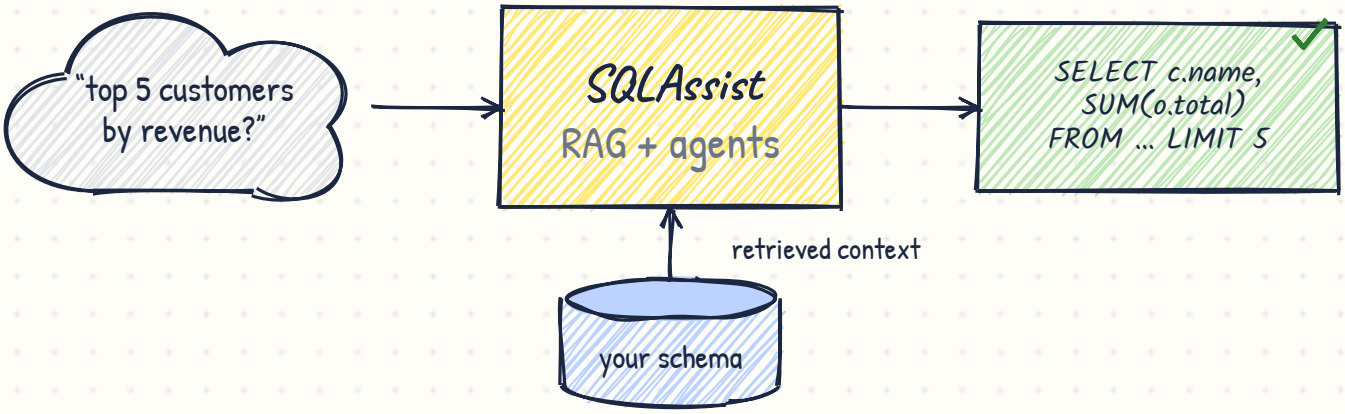
SQLAssist

an AI engineer's notebook

learning LLMs · RAG · Agents · Evaluation · MCP
by building a real SaaS, one phase at a time



How to read this notebook

- highlighter = must remember
- red ink = gotcha / common mistake
- green box = pro tip
- orange card = interview question
- stamps: BUILT ✓ COMING

The deal

Every phase of the project adds pages here. Concepts first (simple → industry level), then *how we actually built it* in this codebase. If someone asks you about a term in this notebook, you should be able to draw its diagram from memory.

Volume 1 · Phase 1 edition · October 2026

repo: github.com/TanmayDeshpande29/AI-powered-SQL-Assistant

What's inside

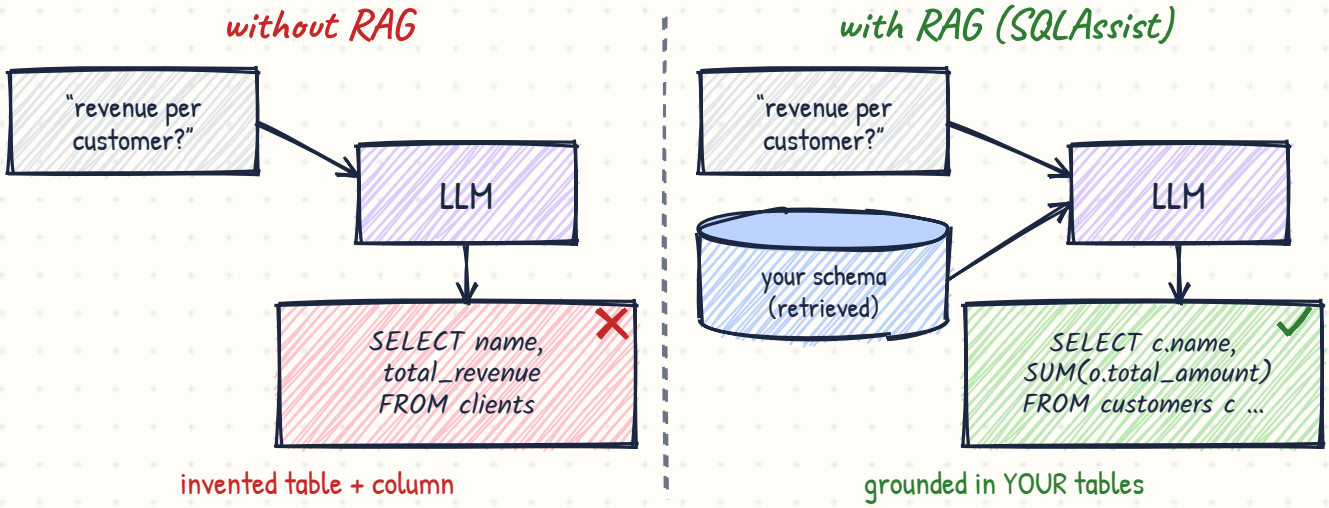
1 · The big picture 3
Target architecture 4
2 · Web backend foundations ✓ 5
FastAPI, ASGI & Pydantic 6
async / await & the event loop 7
Layered architecture & dependency injection 8
3 · The data layer ✓ 9
Connecting to PostgreSQL 10
ORM, sessions & connection pools 11
Migrations & indexes 12
4 · Security basics ✓ 13
JSON Web Tokens (JWT) 14
Access + refresh token flow 15
IDOR, CORS & token storage 16
5 · LLM fundamentals ✓ 17
Prompts, messages & Groq models 18
LangChain & LCEL chains 19
Structured output 20
Reasoning models, tokens & cost 21
Streaming with Server-Sent Events 22
Retries, fallbacks & rate limits 23
Observability: tracing with LangSmith 24
Validating output: ASTs & prompt versions 25
6 · RAG → 26
The two RAG pipelines 27
Chunking: schema-aware table cards 28
Embeddings & similarity 29
Vector databases: Qdrant & HNSW 30
Retrieval: BM25, dense, hybrid (RRF) 31
Reranking, query rewriting & FK expansion 32
7 · RAG evaluation 33
Retrieval metrics, worked by hand 34
Generation & text-to-SQL metrics 35
8 · Agents 36
ReAct & LangGraph 37
Our agent team 38
9 · Memory management 39
10 · Human-in-the-loop 40
11 · MCP 41
MCP protocol, transports & lifecycle 42
12 · Guardrails, shipping & scale 43
Docker, CI/CD, scaling & observability 44
Glossary A–L 45
Glossary M–Z 46
Phase journal · 0: Foundation ✓ 47
Phase journal · 1: LLM layer ✓ 48

OVERVIEW

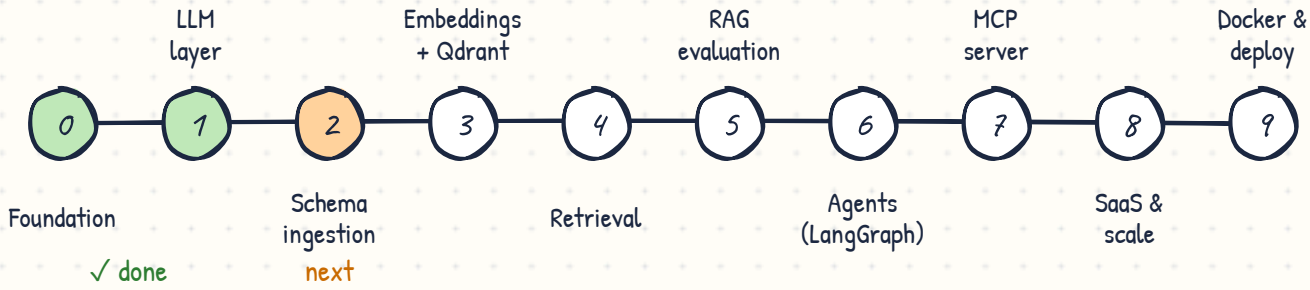
1 What are we building?

The problem. Ask a general-purpose LLM *"revenue per customer?"* and it happily writes SQL against tables it has never seen: it guesses names like `clients.total_revenue`. It knows SQL, but not *your* database.

Our fix. At question time, retrieve the relevant part of the user's schema and give it to the model (that's **RAG**), then let a small **team of agents** validate, fix, secure and explain the SQL.



The 10-phase journey

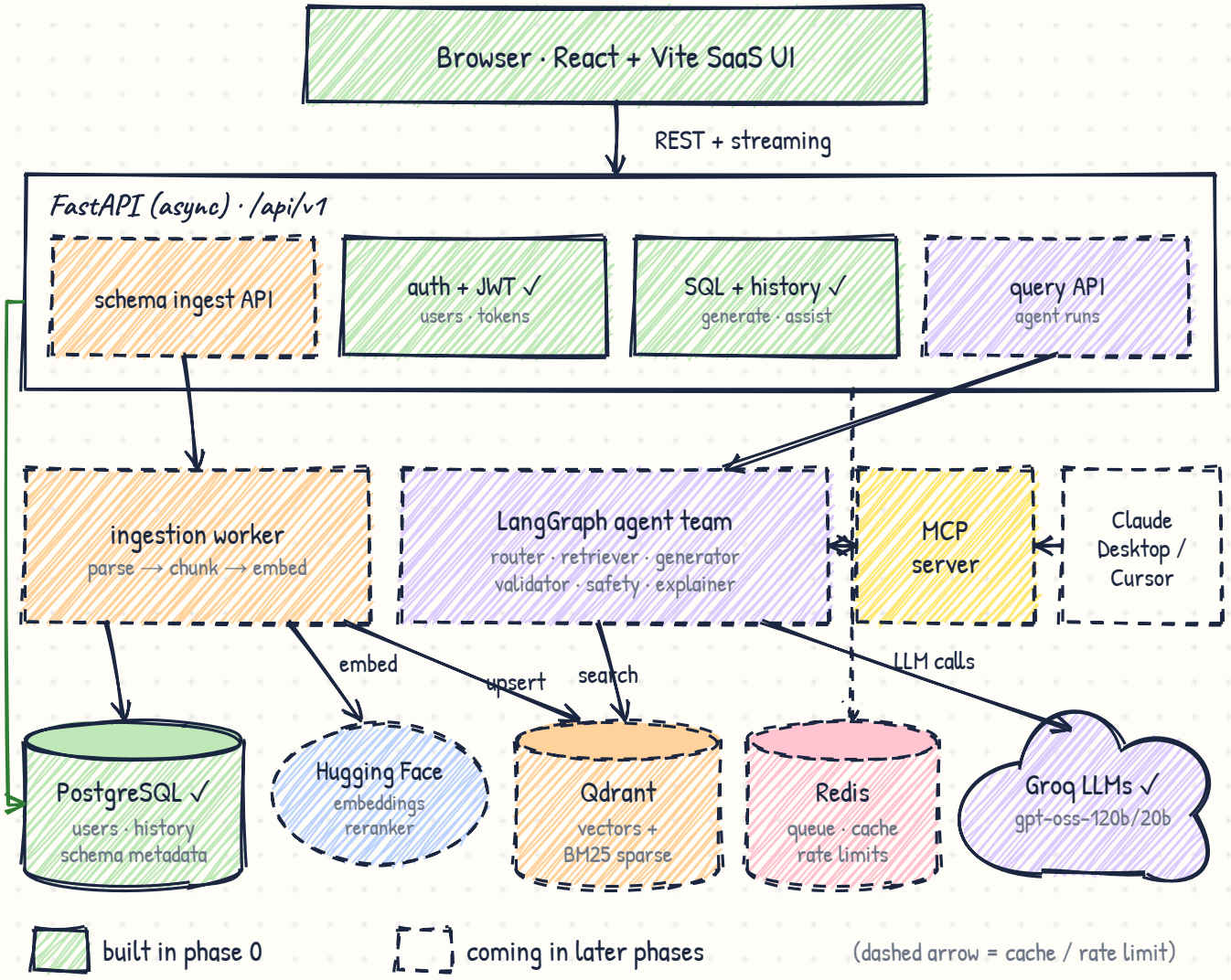


✓ *PRO TIP* Each phase = a working, tested increment. We never build phase N+1 on a broken phase N.

Stack: React · FastAPI · PostgreSQL · Qdrant · Groq (gpt-oss) · Hugging Face embeddings · LangChain + LangGraph · MCP · Docker

OVERVIEW

Target architecture (where we're heading)



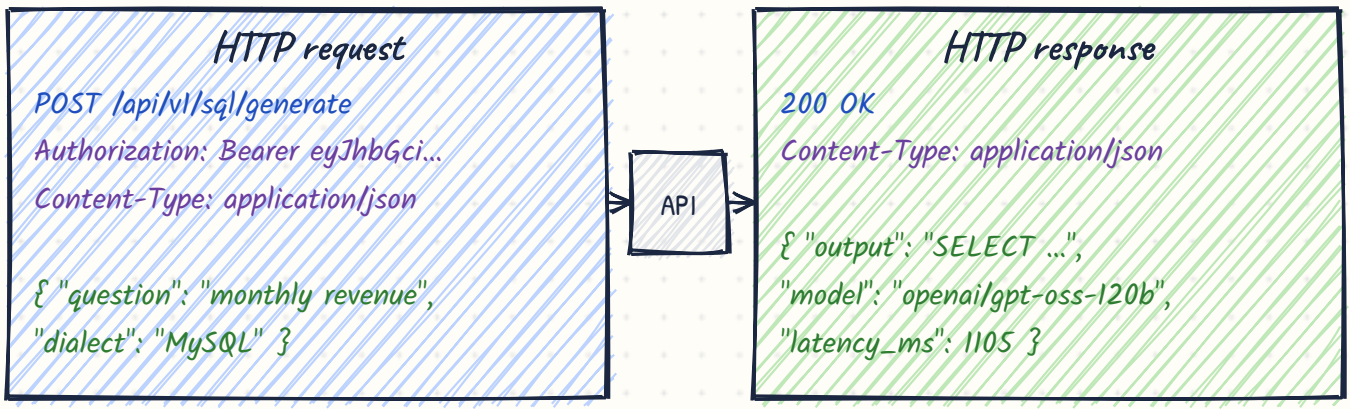
Part	Job in SQLAssist
PostgreSQL	source of truth: users, query history, uploaded schema metadata, eval results
Qdrant	vector database: embeddings of schema "table cards" for semantic + keyword search
Groq	runs the LLMs (gpt-oss) very fast on custom LPU chips; free tier with rate limits
Hugging Face	open embedding + reranker models (run locally in our worker)
LangGraph	orchestrates the agent team as a state graph (loops, branches, memory, human approval)
MCP server	exposes our tools so *other* AI apps (Claude Desktop, Cursor) can use SQLAssist

BUILT ✓ phase 0

2 Web backend foundations

HTTP in one picture

The browser and our API talk in ***requests*** and ***responses***. Every request has a method + path, headers (metadata) and an optional body (data, here JSON).



Status codes we actually use

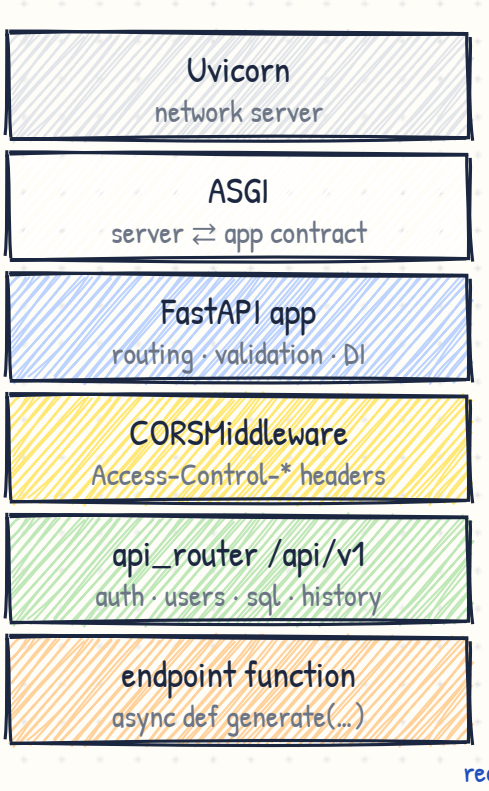
Code	Meaning	When SQLAssist returns it
200 / 201 / 204	OK / Created / No Content	success · signup · delete or password change
400	Bad Request	wrong *current* password on change-password
401	Unauthorized = ***not authenticated***	missing / expired / forged token, bad login
403	Forbidden = authenticated but ***not allowed***	(phase 8: workspace roles)
404 / 409	Not Found / Conflict	row missing *or not yours* · email already registered
422	Unprocessable Entity	Pydantic validation failed (field errors listed)
500 / 502 / 503	our bug / upstream failed / not ready	unhandled error · Groq down · DB down (readiness)

Interview Q: 401 vs 403?

A: 401: "I don't know who you are" (authentication failed). 403: "I know who you are, and you may not do this" (authorization failed).

⚠ **GOTCHA** The old API returned **200** with `{"error": ...}` on failures, so the frontend couldn't use `response.ok`. Now every error has a real status + one JSON envelope: `{"error": {code, message, details}}`.

The server stack



→ ***Uvicorn***: the actual network server. Opens the port, parses raw HTTP bytes.

→ ***ASGI*** (Asynchronous Server Gateway Interface): the standard "plug shape" between async servers and Python apps. Successor of WSGI (sync).

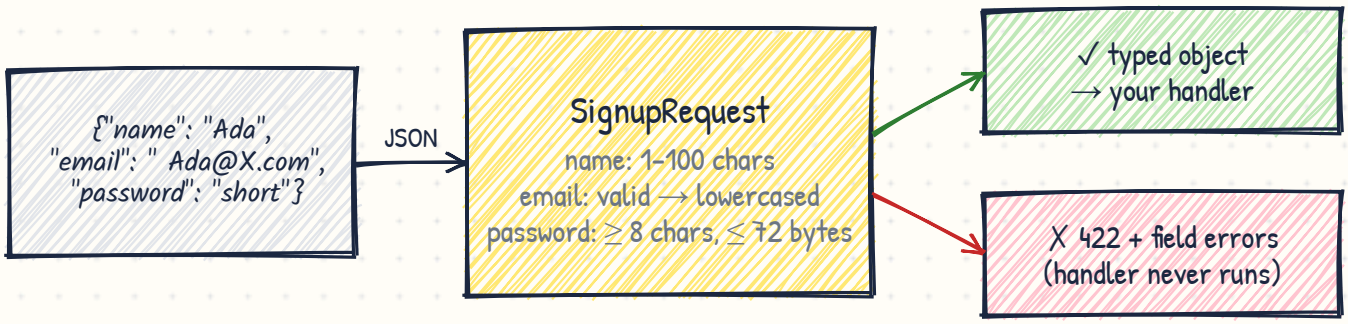
→ ***FastAPI*** = ***Starlette*** (routing, middleware, requests) + ***Pydantic*** (validation) + type hints → automatic OpenAPI docs at `/docs`.

→ ***Middleware*** wraps every request (CORS headers, later: request IDs, rate limits).

→ ***Router*** groups endpoints: we mount everything under `/api/v1` so v2 can live next to it.

✓ *PRO TIP* Versioned URLs (`/api/v1`) let you change the API without breaking old clients.

Pydantic: the bouncer at the door




```python
class SignupRequest(BaseModel):
    name: str = Field(min_length=1,
                      max_length=100)
    email: NormalizedEmail   # EmailStr + lower()
    password: Password       # 8+ chars, ≤72 bytes
```

Why it matters: the handler can *trust* its inputs. Validation, type conversion and error messages come for free from type hints.

Response models work the other way round: `response_model=UserRead` only lets declared fields out, so `hashed_password` can never leak in a response.

async / await & the event loop

Analogy: one waiter (the event loop thread), many tables (requests). A *sync* waiter stands at the kitchen until each dish is ready. An *async* waiter places the order (`await`), serves other tables, and comes back when the kitchen rings.

sync: blocking call inside async def (the old bug)



async: await frees the loop while waiting



→ `async def` + `await` means "I'm waiting on I/O (network, disk, DB): run something else meanwhile".

→ It only helps I/O-bound work. CPU-heavy work (embedding 10k chunks) still blocks → run it in a worker / process pool.

→ Inside `async def` , use async libraries only: `asyncpg` , `httpx` , LangChain's `ainvoke` .

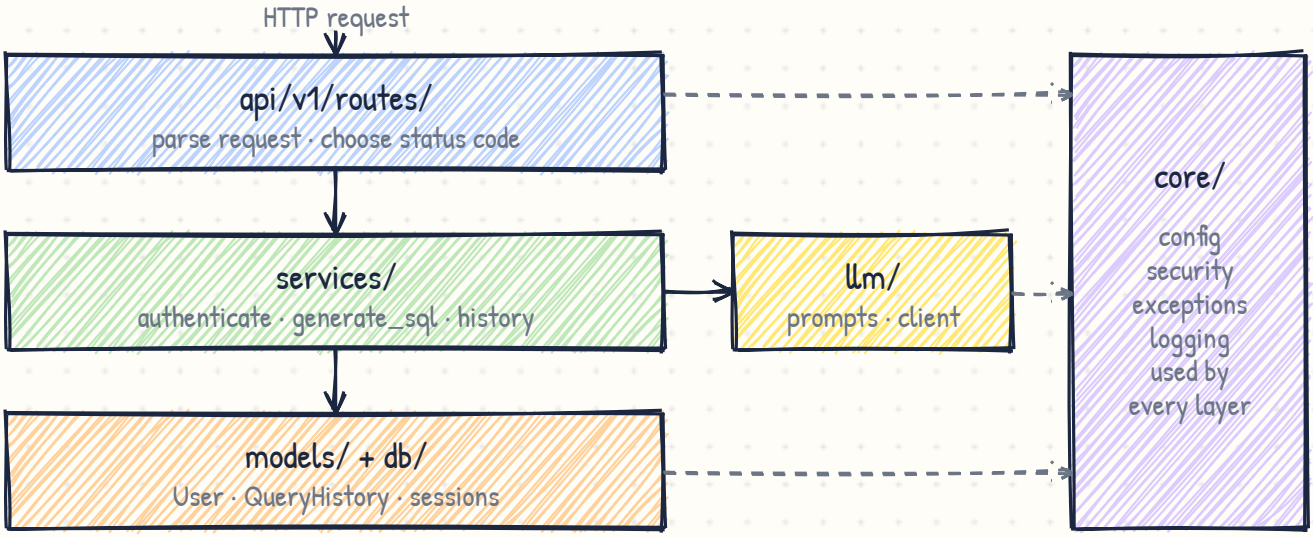
⚠ **GOTCHA** Old code called the **sync** Groq client inside `async def` . One slow LLM call froze the *whole server* for every user.

```
result = await chain.ainvoke(inputs)
# need a blocking lib? offload it:
data = await asyncio.to_thread(fn)
```

Interview Q: Is async the same as multithreading?

A: No. Async is **one thread** switching between tasks at `await` points (cooperative). Threads are scheduled by the OS (preemptive). Async scales to thousands of waiting connections cheaply; it doesn't make CPU work faster.

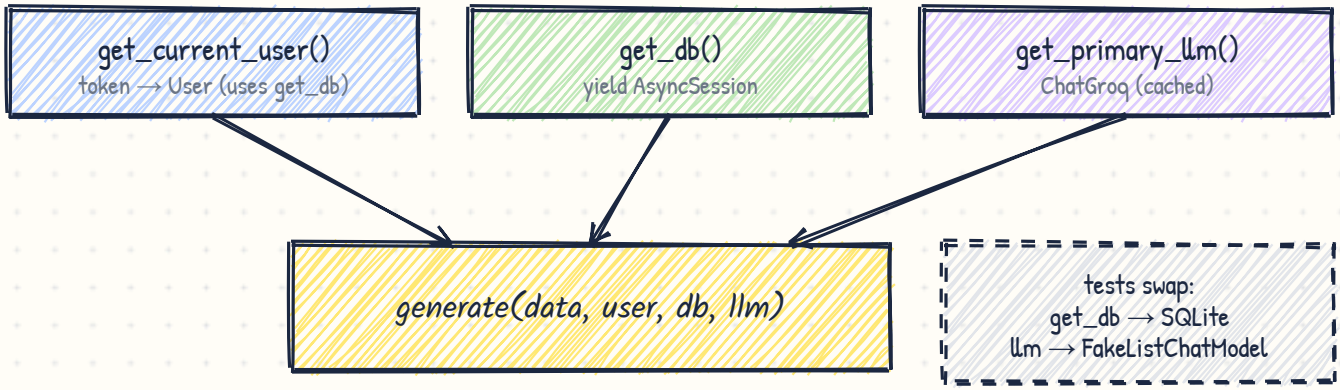
Layered architecture



dependency rule: arrows point down / sideways — lower layers never import upper ones

Why bother? Services don't know about HTTP, so the same function can be called by a route today, by a LangGraph agent in phase 6 and by an MCP tool in phase 7. Tests call services directly too.

Dependency injection with Depends()




```
# api/deps.py: declare once, reuse everywhere
DbSession   = Annotated[AsyncSession, Depends(get_db)]
CurrentUser = Annotated[User, Depends(get_current_user)]
PrimaryLLM = Annotated[BaseChatModel,
                       Depends(get_primary_llm)]
```

Interview Q: Why dependency injection?

A: The function declares *what* it needs, the framework decides *how* to build it. You get swappable implementations (tests, configs), one place for cross-cutting rules (auth), and managed lifecycles (session closed after each request).

BUILT ✓ phase 0

3 The data layer

Why PostgreSQL (and not Mongo) for us

→ Our data is **relational**: users → history; soon workspaces → schemas → tables → columns → foreign keys.

→ **Constraints in the DB** (UNIQUE email, FOREIGN KEY, NOT NULL) hold even when app code has a bug.

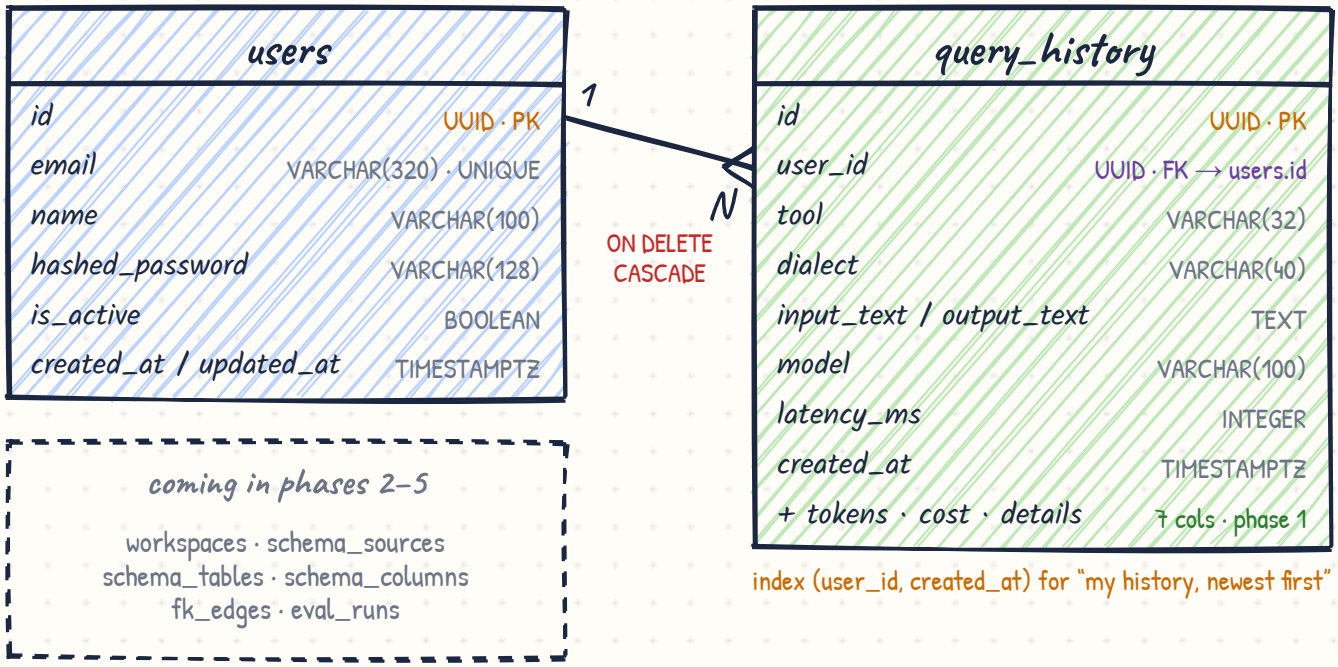
→ **ACID transactions**: all-or-nothing writes.

→ Still flexible: `JSONB` columns, full-text search, extensions.

UUID primary keys instead of 1, 2, 3…

- not guessable (`/history/5` → "let me try 6")
- can be generated anywhere (workers) before insert
- cost: 16 bytes, random B-tree inserts (UUIDv7 fixes ordering)

Our schema today (ER diagram)



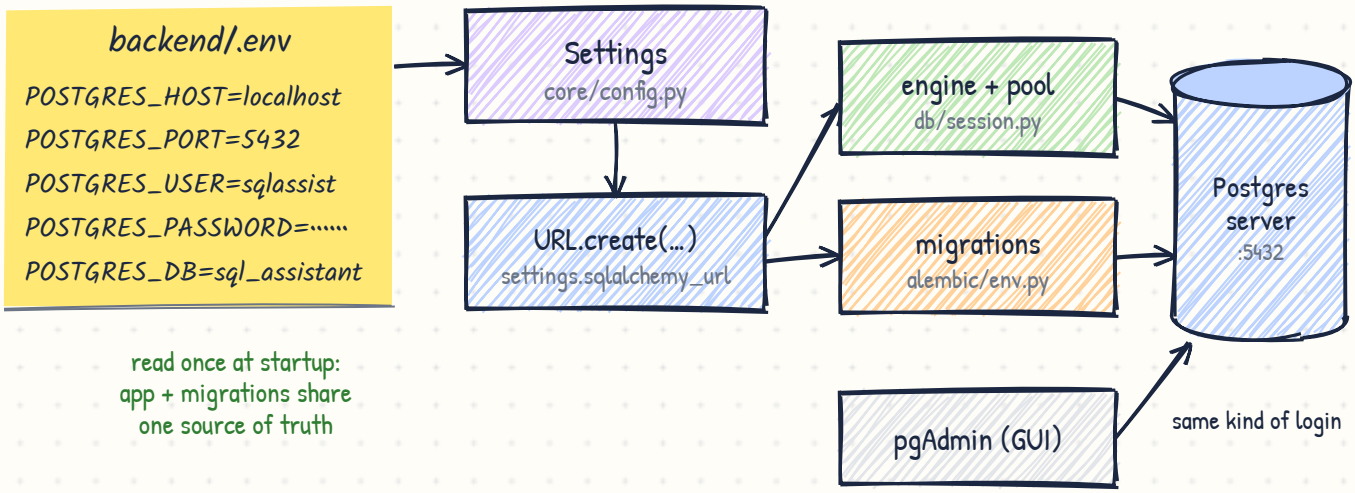
Read the crow's foot: one user (**1**) has many history rows (**N**). Deleting a user deletes their history automatically (`ON DELETE CASCADE`), done *by Postgres*, not by our code.

⚠ **GOTCHA** MongoDB was removed entirely. Its old history route never awaited the async driver, so ***nothing was ever saved.***

BUILT ✓ phase 0

Connecting to PostgreSQL

pgAdmin and our API are both just *clients* of the same PostgreSQL server. They log in with the same kind of credentials; ours live in `backend/.env`, and every part of the backend reads them from one place.



Anatomy of a connection URL



Thing	What it is
PostgreSQL server	the program listening on port 5432 (a Windows service); it holds many databases
pgAdmin	a GUI *client*; it logs into the server exactly like our API does
role (user)	a login on the server; `postgres` is the superuser created at install
database · schema	`sql_assistant` holds schemas; our tables live in the `public` schema

```
-- pgAdmin → Query Tool, ONE statement at a time
CREATE ROLE sqlassist WITH LOGIN PASSWORD '…';
CREATE DATABASE sql_assistant OWNER sqlassist;
```

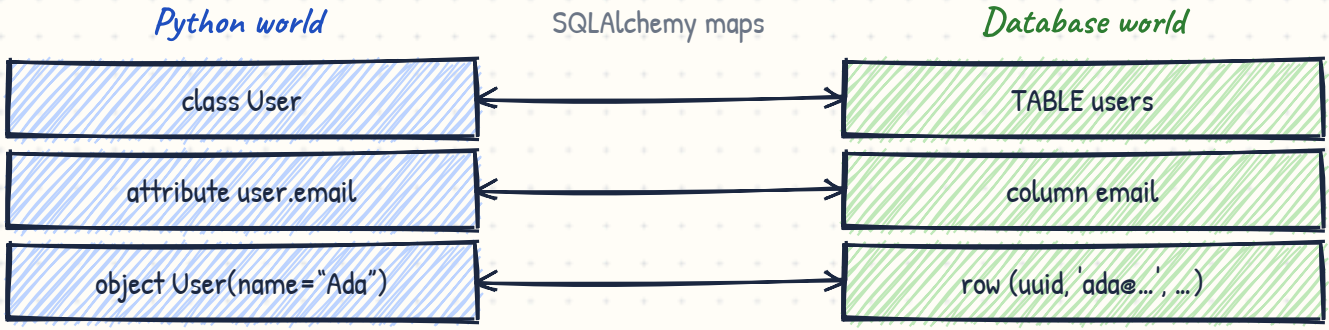
Run them one by one (`CREATE DATABASE` can't run inside a transaction block), then `alembic upgrade head`. Edited `.env`? A running server keeps the old values until it restarts: our dev command restarts on `.env` changes.

⚠ **GOTCHA** **Two passwords, don't mix them up.** pgAdmin's *master password* only encrypts the logins pgAdmin saves; it never goes in `.env`. And a password like `p@ss#1` breaks a hand-written URL (the `@` looks like the host separator), which is why we build it with `URL.create()`.

Interview Q: Why not let the app log in as `postgres`?

A: Least privilege: if the app is compromised (SQL injection, leaked .env), a superuser could drop every database or even run OS commands (`COPY … PROGRAM`). A role that only owns `sql_assistant` limits the damage to that database.

ORM: objects ⇄ rows

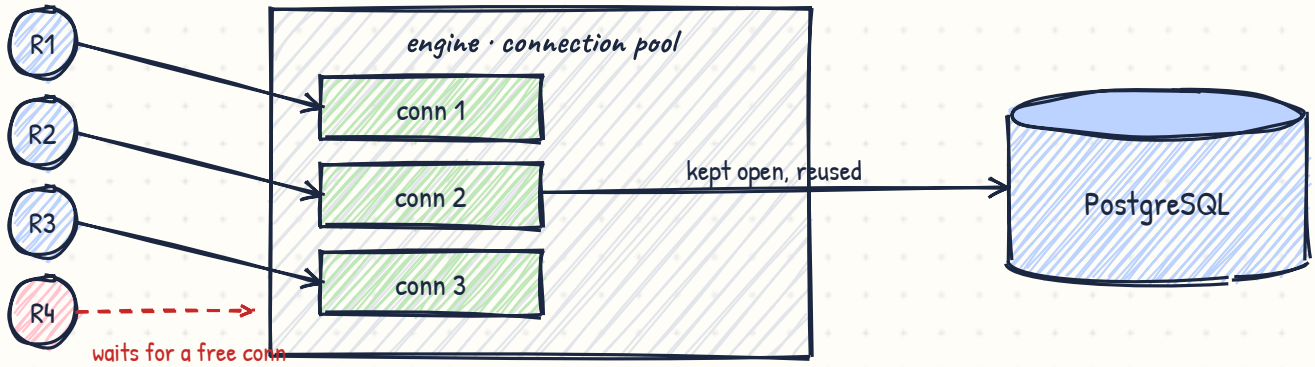



```python
class User(UUIDPrimaryKeyMixin, TimestampMixin, Base):
    __tablename__ = "users"
    email: Mapped[str] = mapped_column(
        String(320), unique=True, index=True)
    queries: Mapped[list["QueryHistory"]] = relationship(
        back_populates="user", passive_deletes=True)
```

Session = unit of work. `db.add(user)` stages it, `await db.commit()` sends the INSERT and commits the transaction. On error: `rollback()`, nothing half-written.

Mixins share columns (`id`, `created_at`) across models without copy-paste.

Connection pool



Opening a Postgres connection costs a TCP + auth handshake. The ***engine*** keeps a pool of open connections; each request *borrows* one and returns it. `pool_pre_ping=True` tests a connection before use (the server may have dropped it).

⚠ **GOTCHA** ***expire_on_commit.*** After commit, SQLAlchemy normally "forgets" loaded values and reloads them lazily. In async code a lazy load can't happen implicitly → `MissingGreenlet` error. We set `expire_on_commit=False`.

Interview Q: *What is the N+1 query problem?*

A: Loading 1 list query, then 1 extra query *per row* for a relationship (N rows → N+1 queries). Fix with eager loading (`selectinload`) or a JOIN.

BUILT ✓ phase 0

Migrations: git for your schema

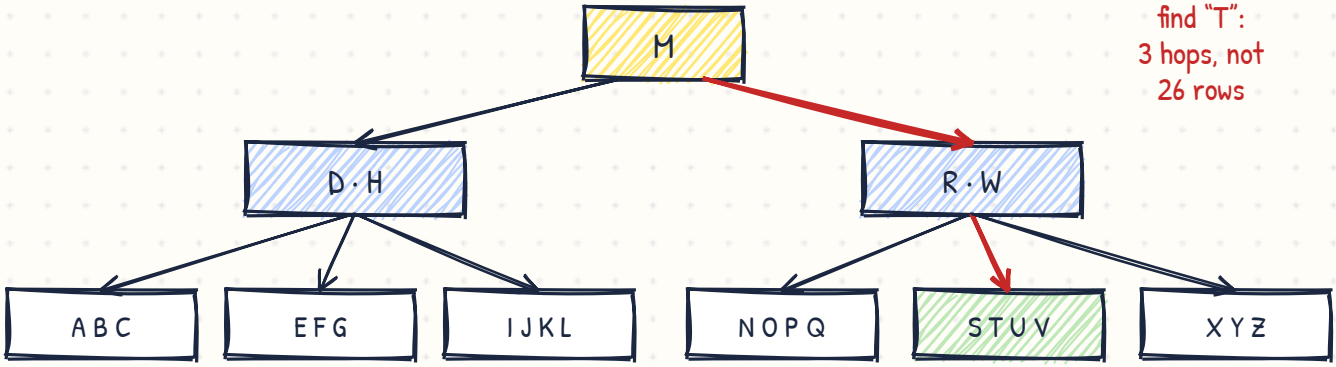


the alembic_version table stores which revision the database is at

```
uv run alembic revision --autogenerate -m "msg"
uv run alembic upgrade head     # apply all
uv run alembic downgrade -1     # undo last
uv run alembic check            # drift?
```

⚠ GOTCHA **Autogenerate is a draft.** Ours was generated against SQLite, so it wrote `(CURRENT_TIMESTAMP)` + SQLite batch ops. We rewrote it by hand to portable SQL. Always read a migration before running it.

Indexes: the phone book trick



→ A ***B-tree index*** keeps keys sorted in a shallow tree → lookups in *O(log n)* instead of scanning every row.

→ ***Composite index*** `(user_id, created_at)` answers `WHERE user_id = ? ORDER BY created_at DESC` in one walk. Leftmost-prefix rule: it helps queries on `user_id` alone, but not on `created_at` alone.

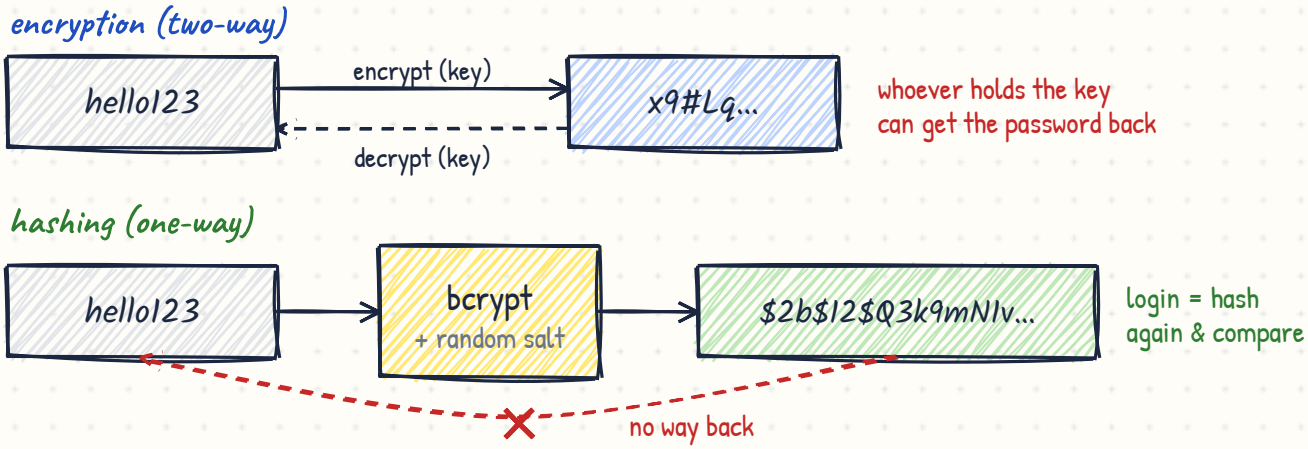
→ ***Unique index*** on email: the DB rejects duplicates even if two signups race; we catch `IntegrityError` → 409.

→ Cost: faster reads, slower writes + disk. Index what you ***filter, join and sort*** on.

BUILT ✓ phase 0

4 Security basics

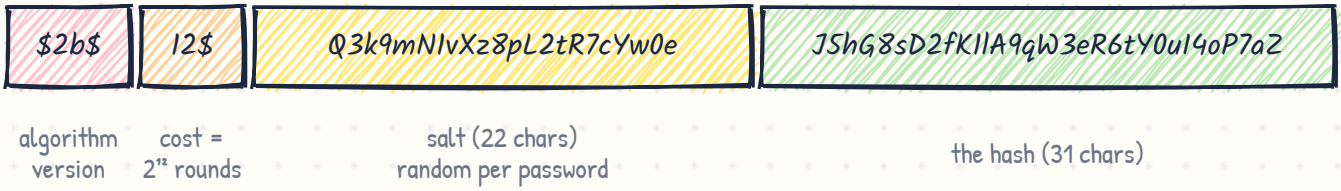
Hashing ≠ encryption



→ We never need the original password, only to *check* it → store a ***hash***. A leaked DB ≠ leaked passwords. (The old code stored plain text.)

→ ***Salt***: random bytes mixed in per password. Same password → different hashes → precomputed "rainbow tables" are useless. Our test asserts `hash("x") != hash("x")`.

→ ***bcrypt is slow on purpose***: cost 12 = 2^{12} rounds (~0.25 s). Fine for one login, brutal for billions of guesses. Fast hashes like SHA-256 are wrong for passwords.

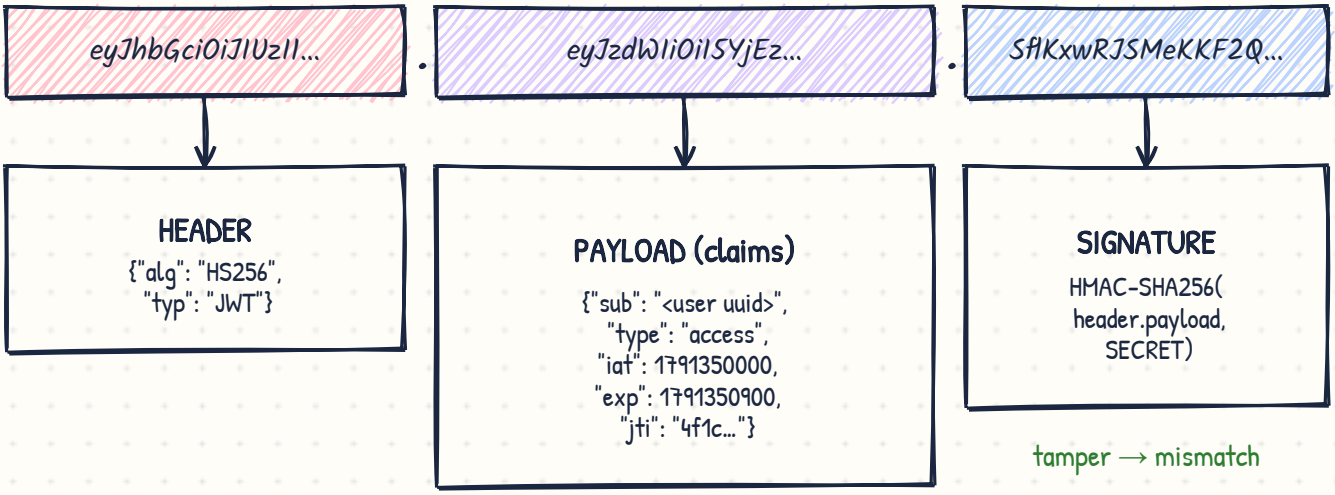


⚠ GOTCHA **72-byte limit.** bcrypt ignores bytes after 72 (and bcrypt 5 raises an error). We reject longer passwords, counting *bytes*, not characters: one emoji = 4 bytes.

⚠ GOTCHA **User enumeration.** If "unknown email" answers faster than "wrong password", attackers learn who has an account. We return the same message and still run bcrypt against a dummy hash.

BUILT ✓ phase 0

Anatomy of a JWT



base64url-encoded — NOT encrypted!

→ **Signed, not secret:** anyone can decode the payload (try jwt.io). Never put secrets in it.

→ The server recomputes the signature with its **SECRET**. Change one character of the payload → signature mismatch → rejected.

→ **Stateless:** verifying needs no DB lookup (we still load the user to check `is_active`).

→ **Claims:** `sub` who · `exp` expiry · `iat` issued-at · `jti` unique id (enables revocation lists) · `type` our own: *access* or *refresh*.

⚠ GOTCHA **Weak secret.** HS256 needs ≥ 256 bits of randomness. Ours was 13 characters → rotated to `secrets.token_urlsafe(48)`, and config now refuses < 32 chars at startup.

⚠ GOTCHA **Pin the algorithm.** `jwt.decode(..., algorithms=["HS256"])` blocks the classic `"alg": "none"` and algorithm-confusion attacks.

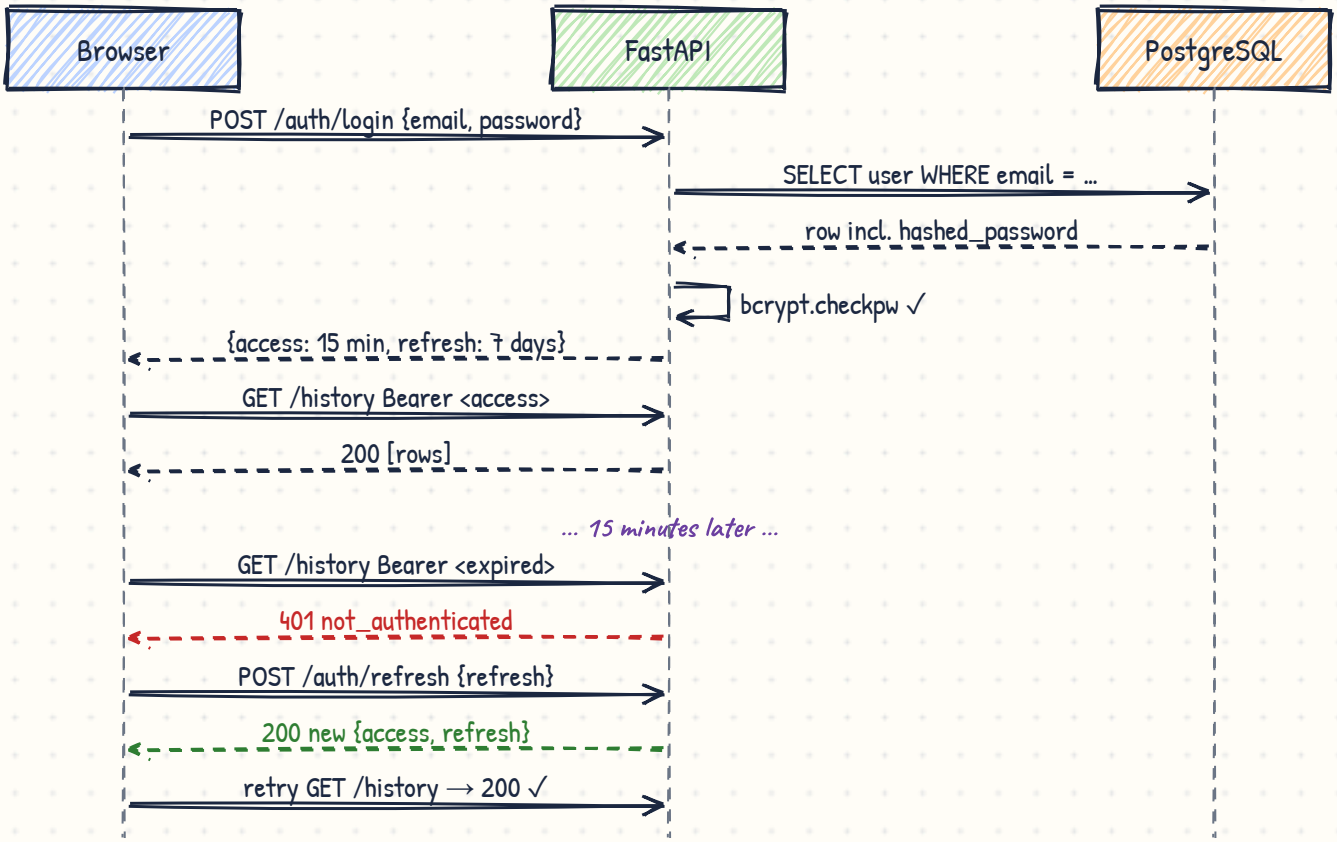
```
# core/security.py (PyJWT)
payload = jwt.decode(token, SECRET,
    algorithms=["HS256"],
    options={"require": ["exp", "sub", "type"]})
if payload["type"] != expected_type:
    raise jwt.InvalidTokenError()
```

Interview Q: **Why did we add a "type" claim?**

A: Without it, a long-lived **refresh** token would also work as an **access** token on every endpoint, defeating short expiry. A test checks this can't happen.

BUILT ✓ phase 0

Access + refresh tokens, step by step



Why two tokens? The access token travels on *every* request, so it's short-lived: if stolen, damage lasts minutes. The refresh token is sent to *one* endpoint only, rarely.

Single-flight refresh (`frontend/src/lib/api.js`): when 4 requests hit 401 at once, they all await one shared refresh call. We saw exactly this in the browser network log during testing.

```
// api.js: share one refresh between callers
let refreshInFlight = null
function refreshTokens() {
  refreshInFlight ??= fetch("/auth/refresh", …)
    .then(saveTokens)
    .finally(() => (refreshInFlight = null))
  return refreshInFlight
}
```

Interview Q: JWTs are stateless. How do you log someone out before the token expires?

A: You can't "un-sign" a token, so you add a little state back:

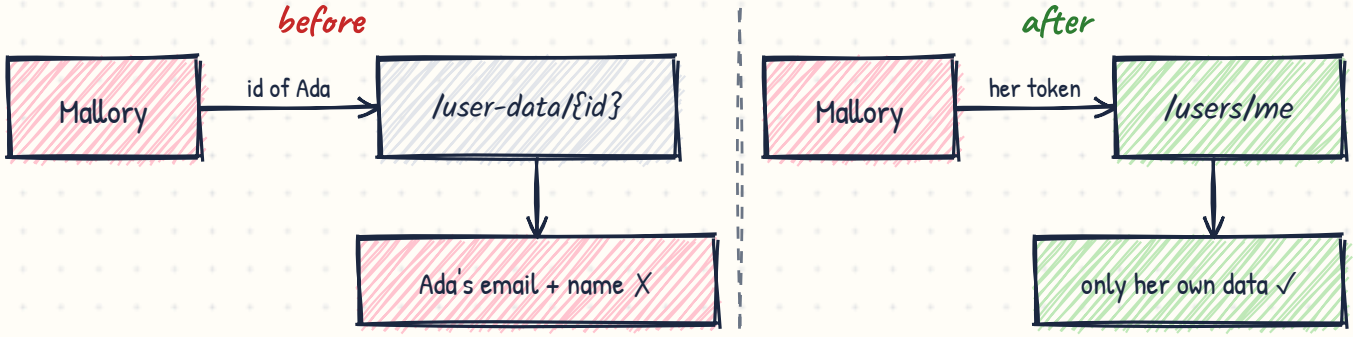
- ***short access tokens*** (ours: 15 min) bound the damage window;
- ***denylist*** the token's `jti` in Redis until its `exp` , checked in `get_current_user` ;
- ***refresh-token rotation***: each refresh issues a new refresh token and invalidates the old one; if an old one is ever reused, it was stolen, so revoke the whole family;
- ***token version*** on the user row: bump it on password change → every older token fails.

✓ PRO TIP This is exactly why our tokens already carry a `jti` (unique id): revocation can be added in phase 8 without changing the token format.

BUILT ✓ phase 0

IDOR: the bug we had

Insecure Direct Object Reference: the API trusts an id from the URL without checking ownership. The old `GET /user-data/{user_id}` returned *any* user's data to *anyone*, no token needed.

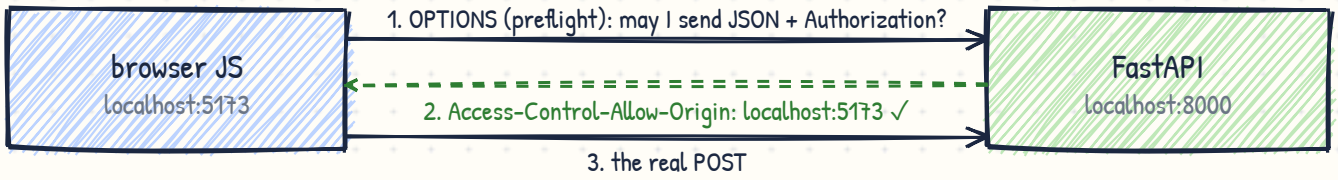


→ Identity comes from the **verified token**, never from the URL or body.

→ Every query is scoped: `WHERE user_id = :current_user`. Test: Bob cannot list or delete Alice's history.

→ For someone else's row we return **404, not 403**: don't even confirm it exists.

CORS (Cross-Origin Resource Sharing)



Different port = different **origin**. Browsers block JS from reading cross-origin responses unless the server allows that origin (`CORS_ORIGINS` setting). Requests with JSON or an `Authorization` header first send an `OPTIONS` **preflight**. CORS protects users' browsers; it is not API authentication (curl ignores it).

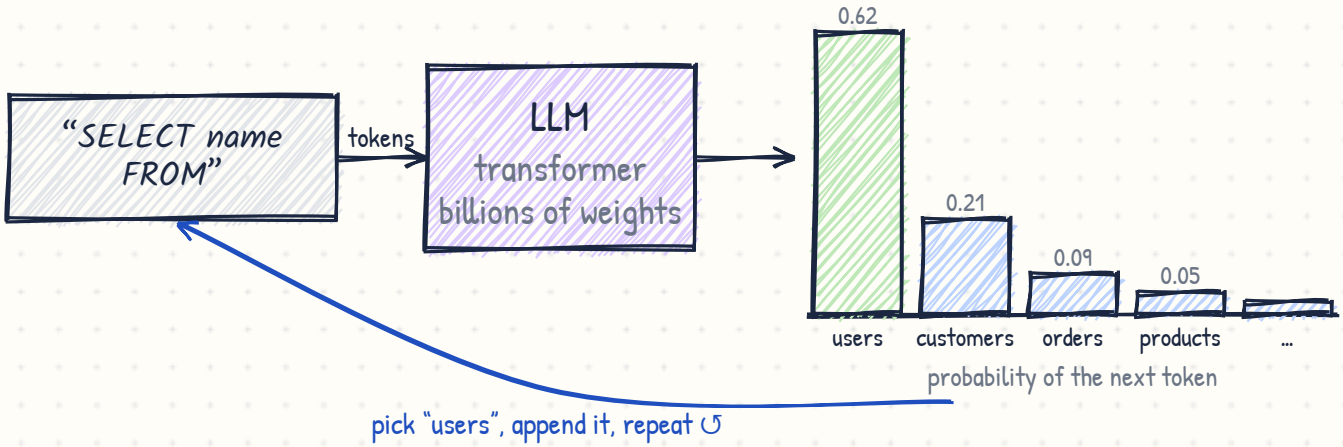
Where should the browser keep tokens?

Option	+	−
localStorage (now)	simple, works with any API host	readable by injected JS (XSS)
httpOnly cookie (phase 8)	JS can't read it → XSS can't steal it	needs CSRF protection + same-site setup

BUILT ✓ phases 0–1

5 LLM fundamentals

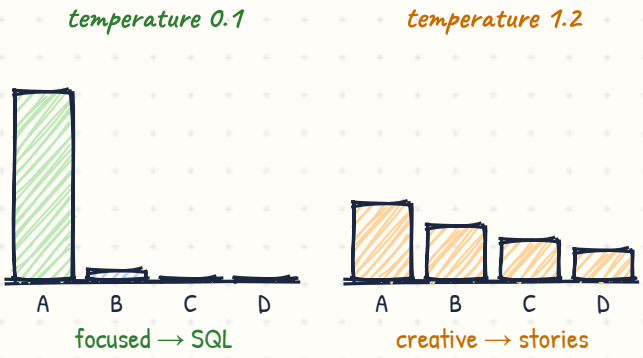
An LLM predicts the next token



→ ***Token*** = a chunk of text (≈ 4 characters of English). Models read, bill and limit in tokens.

→ ***Autoregressive***: generate one token, append, run again. That's why long answers are slower.

→ ***Context window*** = max tokens seen at once (input + output). gpt-oss: ~131k. System prompt, schema, history and answer all compete for it.



Temperature reshapes the probability distribution before sampling: low = almost always the top token (we use `0.1` for SQL), high = more random/creative.

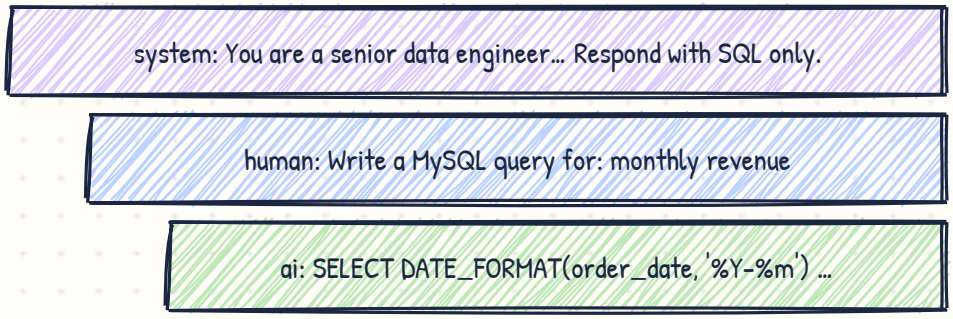
⚠ GOTCHA ***Hallucination*** = fluent but false output (an invented column). You don't "fix" it inside the model; you reduce it with ***grounding*** (RAG), ***validation*** (sqlglot) and ***checking agents***.

Interview Q: ***Same prompt, different answers. Why?***

A: Sampling: with temperature/top-p > 0 the next token is drawn at random from the distribution. Even at 0, batching and GPU math can add tiny non-determinism.

BUILT ✓ phase 0

Chat messages & prompt templates




```
GENERATE_SQL_PROMPT = ChatPromptTemplate.from_messages([
  ("system", "You are a senior data engineer who
    writes correct {dialect} SQL … SQL only."),
  ("human", "Write a {dialect} query for:
    {question}"),
])
```

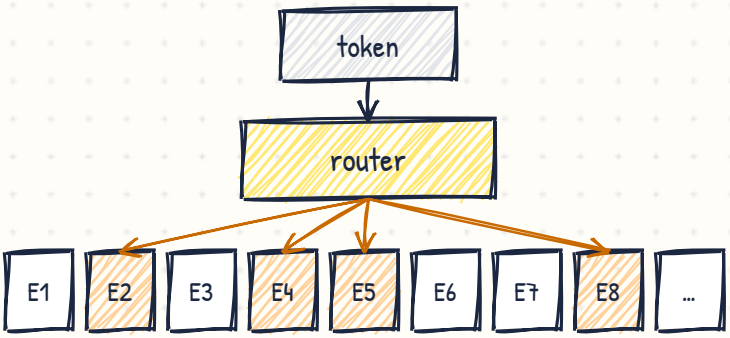
→ ***Placeholders*** are filled by LangChain → prompts are versioned, reusable, testable.

→ Keep user text in the ***human*** message; instructions live in ***system***.

→ Our rules: a role, hard constraints, an escape hatch ("put assumptions in `--` comments").

⚠ **GOTCHA** The old "explain" mode used a system prompt saying *"Output only SQL code"*, then asked for an explanation. Contradictory prompts = unpredictable output. Now each mode has its own template.

Groq & our two model tiers



top-4 of 128 experts run per token

Groq runs open models on custom ***LPU*** chips (Language Processing Units) → very high tokens/second, generous free tier with rate limits.

primary `openai/gpt-oss-120b` : SQL generation & reasoning.
fast `openai/gpt-oss-20b` : routing, rewriting, judging.

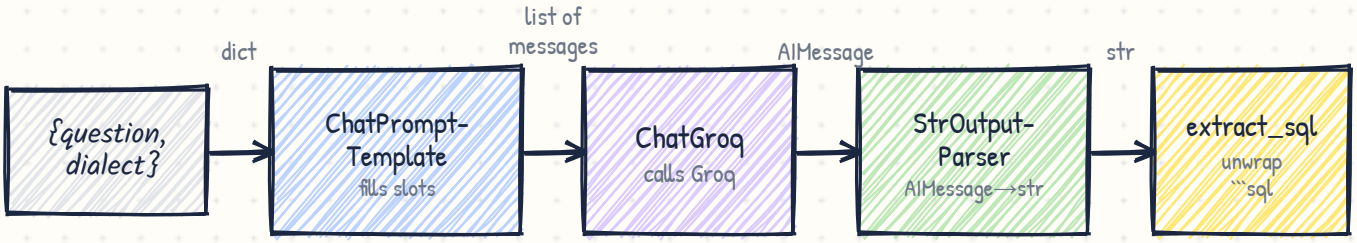
Both are ***Mixture-of-Experts*** reasoning models: huge knowledge, only a few experts compute each token (~5B of ~117B params active for the 120b).

⚠ **GOTCHA** The old code used `llama3-70b-8192`, which Groq ***deprecated***, so *every* AI call failed. Model names now live in config (`LLM_MODEL_PRIMARY`), and we checked the live model list via the API.

BUILT ✓ phase 0

LangChain: chains of Runnables

chain = prompt | llm | StrOutputParser()



every block is a Runnable: .invoke() · .ainvoke() · .stream() · .batch()

→ **LCEL** (LangChain Expression Language): the `|` operator composes Runnables into a *RunnableSequence*; output of one = input of the next.

→ **Why LangChain?** Swap providers (ChatGroq ⇄ ChatOpenAI) without rewriting code, standard message/tool types, and **LangGraph** (phase 6) is built on it.

→ **Defensive parsing:** models sometimes ignore "no markdown" and wrap SQL in ```` ```sql ```` fences. *extract_sql()* unwraps them. Phase 1 adds structured output on top (next pages).

```python
# services/sql.py
chain = prompt | llm | StrOutputParser()
try:
    text = await chain.ainvoke(inputs)
except Exception as exc:
    raise LLMServiceError() from exc  # → 502
```

Resilience: *max_retries=2* retries rate limits (429) and server errors with exponential backoff; *timeout=60s* stops hung calls.

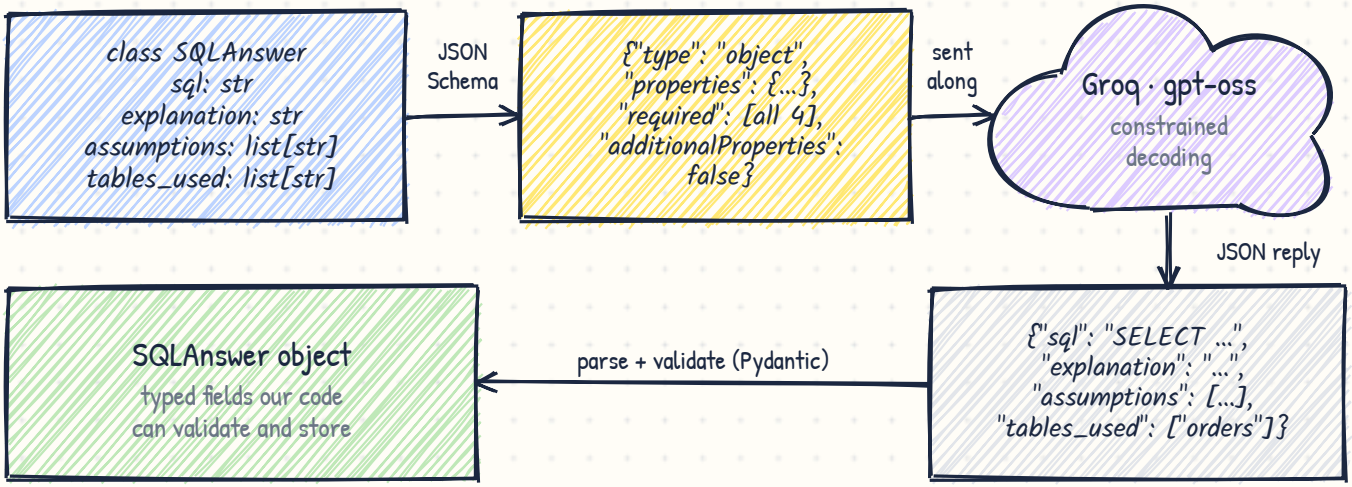
Testing LLM code: *FakeListChatModel* returns scripted replies: fast, free, deterministic. Test the plumbing with fakes; measure the model with evals (chapter 7).

Interview Q: Why not just call the Groq SDK directly?

A: For one call, you could. But we'll need prompt templates, structured outputs, tool calling, streaming, tracing and agent graphs across several providers. LangChain standardises those interfaces, so swapping a model or adding a retriever is one line instead of a rewrite.

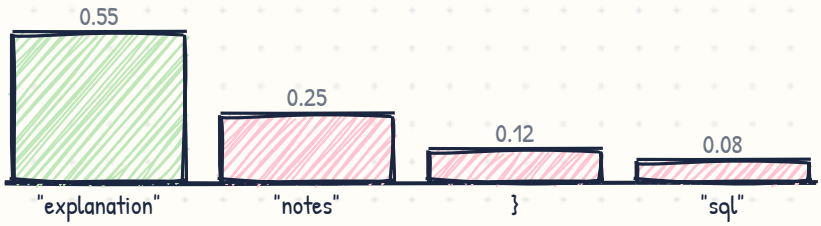
Structured output: make the model fill in a form

Free text is for people; code needs **fields it can trust**. We describe the answer as a Pydantic class, LangChain turns it into a **JSON Schema**, and Groq's decoder is only allowed to produce JSON that matches it.



Method	How it works	Guarantee	SQLAssist
JSON mode	`response_format: json_object`	valid JSON, any shape	not used
Tool calling	schema sent as a tool; the model "calls" it with arguments	usually matches	config option
Strict JSON schema	the decoder masks every token that would break the schema	**always matches**	✓ default

generated so far: {"sql": "SELECT ...", next token?



masked by the schema:

"notes" is not a known key

} — required keys still missing

"sql" — already written

→ sample only from ✓ tokens

```
# services/sql.py
llm.with_structured_output(
    SQLAnswer, method="json_schema",
    strict=True, include_raw=True)
# → {"raw": AIMessage,        ← token usage
#    "parsed": SQLAnswer,     ← typed fields
#    "parsing_error": None}
```

✓ **PRO TIP** The class **docstring and field descriptions travel inside the schema**: they are prompt text the model reads.

⚠ **GOTCHA** Strict mode needs every field *required* and no extra keys. "No assumptions" is `[]`, not a missing field.

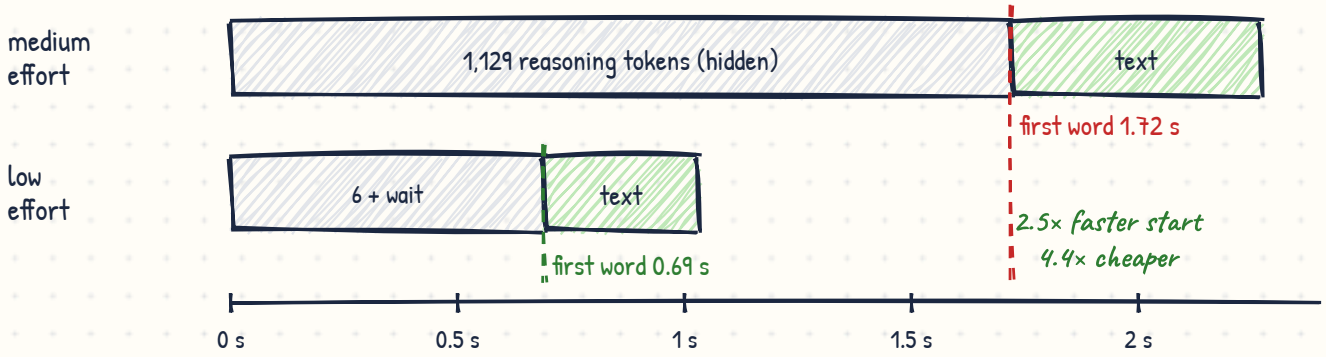
Interview Q: Why not just ask for JSON in the prompt?

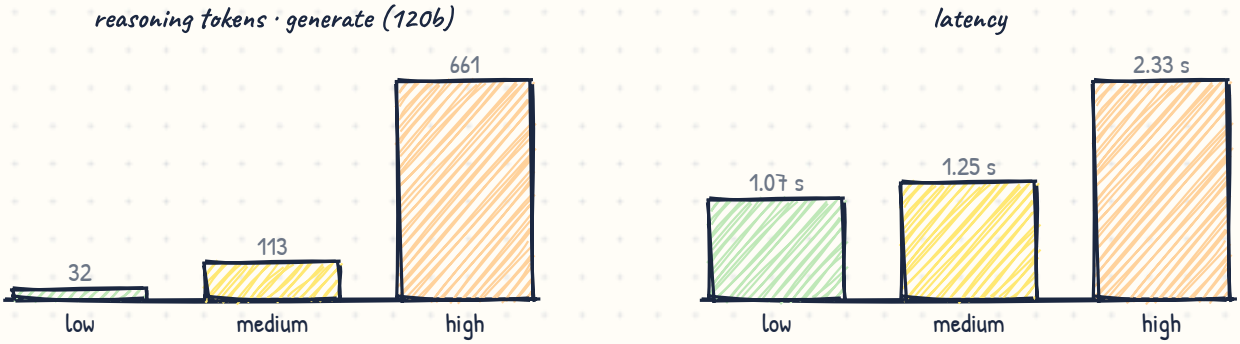
A: A prompt is a request; the model can still add a sentence or forget a field. Constrained decoding makes invalid output *impossible*, so parsing never fails on format.

BUILT ✓ phase 1

Reasoning models think before they answer

gpt-oss first writes hidden ***reasoning tokens*** (a scratchpad), then the visible answer. You never see them, but you wait for them and pay for them. Measured on our explain endpoint:





→ Reasoning tokens ***are output tokens***: billed at the output price and counted toward `max_tokens` (ours: 4096).

→ `reasoning_effort` (low / medium / high) is our per-request ***"Thinking"*** switch. Defaults per prompt: generate = medium (correctness), explain = low (just describing).

→ Raw reasoning is kept away from users: providers advise against showing a model's unfiltered chain of thought. We store only the count.

Cost of one real "generate" call (gpt-oss-120b list prices):

356 input × $0.15 / 1M = $0.0000534
202 output × $0.60 / 1M = $0.0001212
total ≈ $0.000175

≈ $0.17 per 1,000 queries · free tier: $0, but rate-limited.

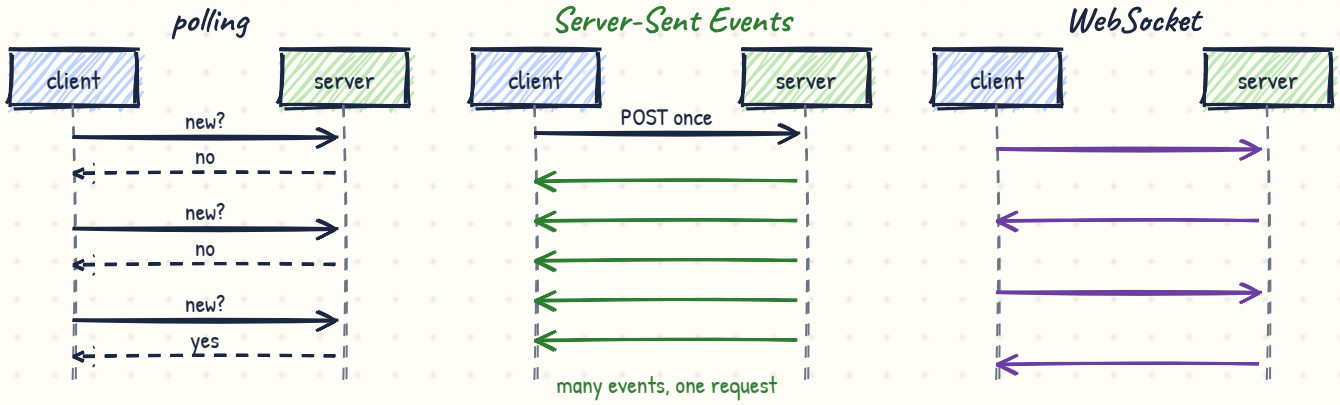
⚠ GOTCHA ***Money is not a float.*** In binary floating point `0.1 + 0.2 = 0.30000000000000004`. Costs use Python `Decimal` and PostgreSQL `NUMERIC(12, 8)`: exact decimal arithmetic.

Interview Q: ***What is time to first token (TTFT), and how did we cut it?***

A: The wait until the first visible word: network + queue + prompt processing + ***hidden reasoning***. Low effort removed ~1,100 reasoning tokens from explanations, so the first word arrives in 0.69 s instead of 1.72 s.

BUILT ✓ phase 1

Streaming: show words as they're written



SSE = one ordinary HTTP response that stays open, with the server writing events into it. One-way, plain HTTP, works through proxies: exactly what token streaming needs. (WebSockets are for two-way traffic, like chat or games.)

```
HTTP/1.1 200 OK
Content-Type: text/event-stream

event: token
data: {"text": "Lists each"}

event: token
data: {"text": " department"}

: ping                ← keep-alive comment

event: done
data: {"output": "...", "usage": {...}}
```

→ **FastAPI:** the route is an `async def` that `yield`s `ServerSentEvent`s, with `response_class=EventSourceResponse`. Keep-alive pings every 15 s come free.

→ **Browser:** `EventSource` can only do GET without headers. We need POST + a Bearer token, so we read `fetch()`'s body stream and split events at blank lines (network chunks don't line up with events).

→ **Cancel:** an `AbortController` stops the fetch; the server notices the disconnect and stops generating.

⚠ GOTCHA The **200 status is sent with the first byte**. Auth and validation run before the stream starts (normal 401/422 JSON); anything that fails later can only be an `event: error`.

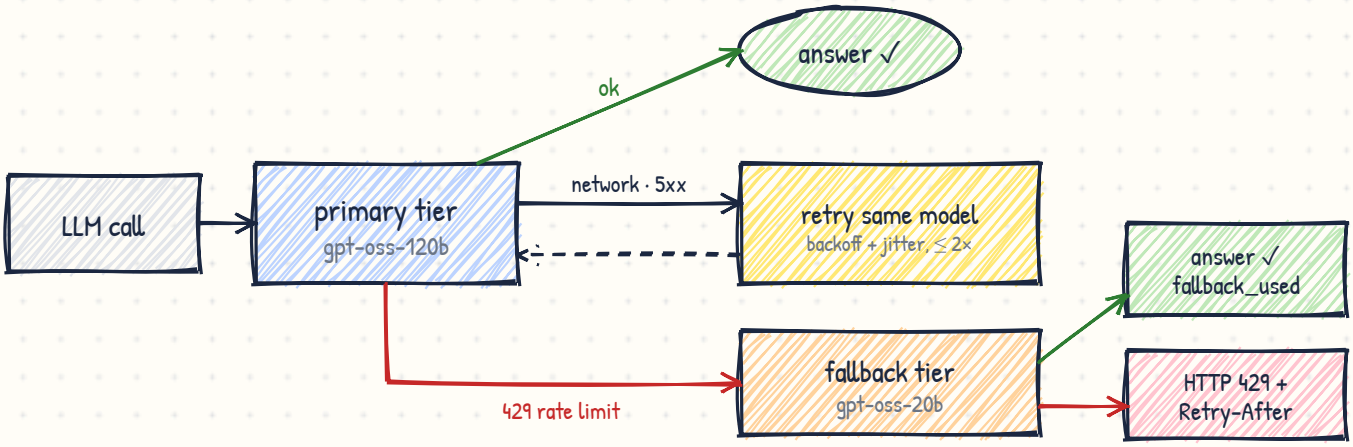
✓ PRO TIP **Our rule:** structure what code consumes (generate, debug, optimize: validated `SQLAnswer`), stream what people read (explain: markdown prose).

Interview Q: Why stream at all if the total time is the same?

A: Perceived latency. Reading can start after ~0.7 s instead of after the full answer, and a long answer shows visible progress instead of a frozen spinner.

BUILT ✓ phase 1

When the LLM fails: retry or fall back?

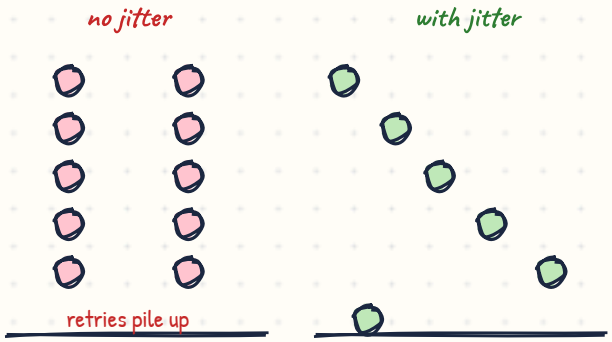


→ ***Retry*** the same thing when the failure is probably temporary: dropped connection, timeout, provider 5xx.

→ ***Fall back*** to something else when waiting won't help soon: rate limit (429), one model's outage.

→ ***Exponential backoff***: wait 0.5 s, 1 s, 2 s… so a struggling server gets room to recover.

→ ***Jitter*** (randomness) spreads clients out so they don't retry in synchronized waves: the *thundering herd*.



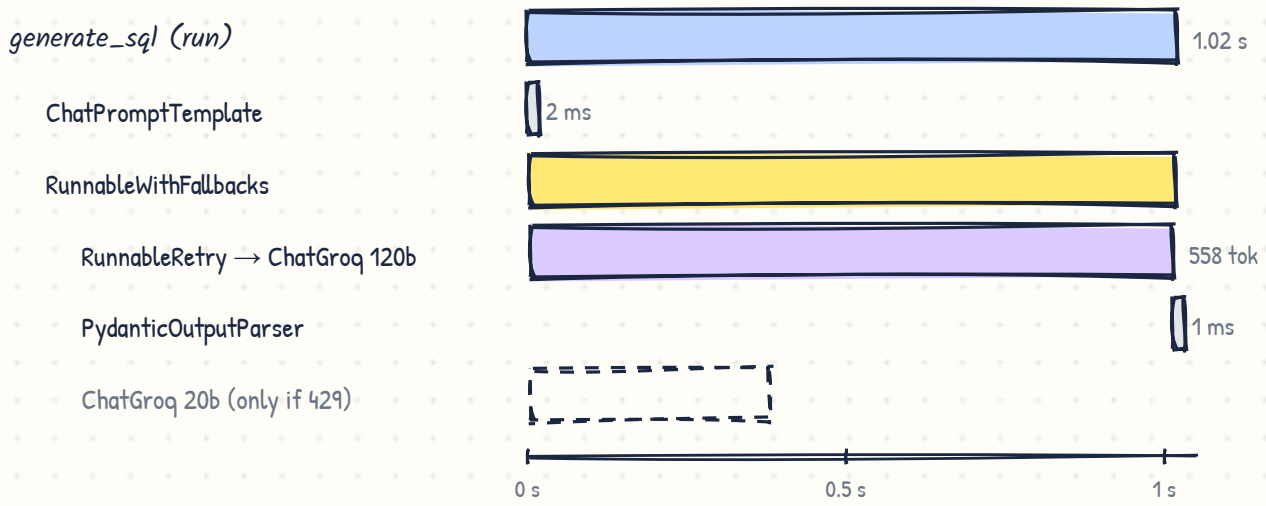
⚠ GOTCHA ***The hidden retry we found.*** By default the Groq SDK retries a 429 by itself, sleeping for the server's `Retry-After` (up to 60 s) *each* time. With 2 retries, one request could hang ~2 minutes before our code ran. Fix: `ChatGroq(max_retries=0)` + LangChain `with_retry(TRANSIENT_ERRORS)` + `with_fallbacks(exceptions_to_handle=(RateLimitError,))`. Lesson: know your SDK's defaults.

Failure	What we do	The client sees
network error / timeout / 5xx	retry the same model (backoff + jitter)	a normal answer, a bit later
429 on the primary	switch to the fast tier immediately	answer + "fallback model" badge
429 on both tiers	give up quickly	**429** + `Retry-After` / SSE `error`
unparseable or empty answer	log it	**502** `llm_unavailable`

BUILT ✓ phase 1

Tracing: see inside every LLM call

A **trace** is the tree of steps one request went through, each with timing, inputs and outputs (a *span* or *run*). For LLM apps it answers "what exactly did we send, what came back, and where did the time go?"



What we record for every call	history row (PostgreSQL)	LangSmith trace
prompt version, model, reasoning effort	✓	✓ (tags + metadata)
tokens (input / output / reasoning), latency, cost	✓	✓ per step
full prompt messages, JSON schema, raw reply	–	✓
retries and fallbacks as child runs	fallback flag	✓
user id	✓ (owner)	✓ (metadata, filterable)

Turn it on: free account at smith.langchain.com → API key → in `backend/.env` set `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY`. LangChain then traces every chain automatically, and LangGraph later traces agents too.

⚠ GOTCHA The LangSmith SDK **caches environment variables on first read**, and pydantic-settings doesn't export `.env` to the environment. So `configure_tracing()` copies them once at startup, before any LLM call.

⚠ GOTCHA **Privacy:** traces contain prompts and answers, so user questions and SQL leave your server. Fine for development; production needs consent, redaction, or a self-hosted tracer (Langfuse).

Interview Q: Why tracing, not just logs?

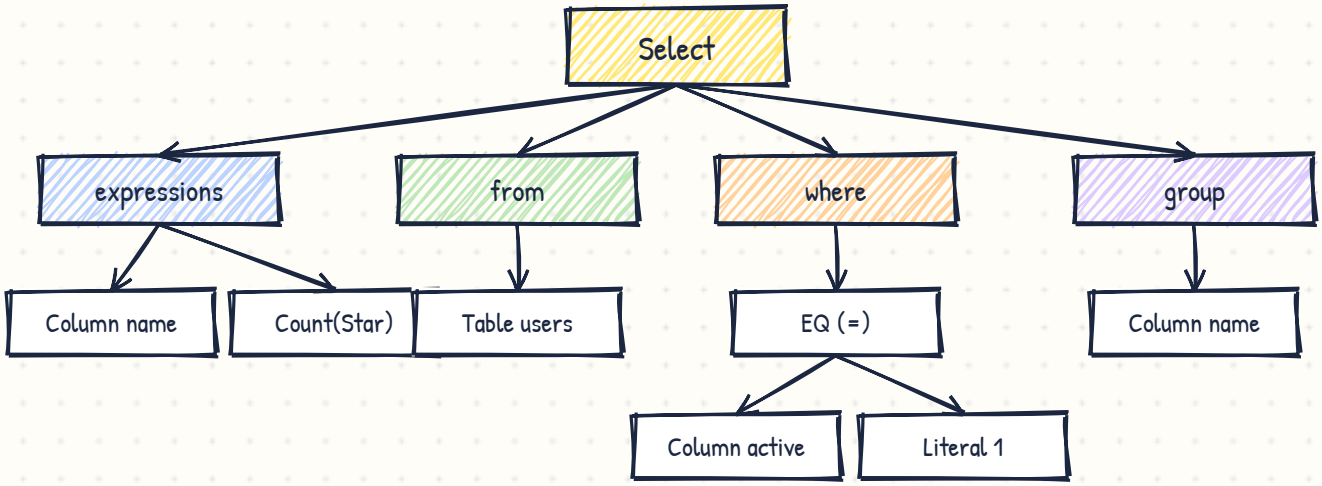
A: Logs are flat lines; a trace keeps the *causal tree* (which retry, which fallback, which prompt version) with timing per step: what you need to debug multi-step chains and, later, agents.

BUILT ✓ phase 1

Never trust model output: parse it

sqlglot turns SQL text into an **AST** (abstract syntax tree): typed nodes instead of characters. If the text can't be parsed for the chosen dialect, it isn't valid SQL, and sqlglot says *where* it broke.

SELECT name, COUNT(*) FROM users WHERE active = 1 GROUP BY name

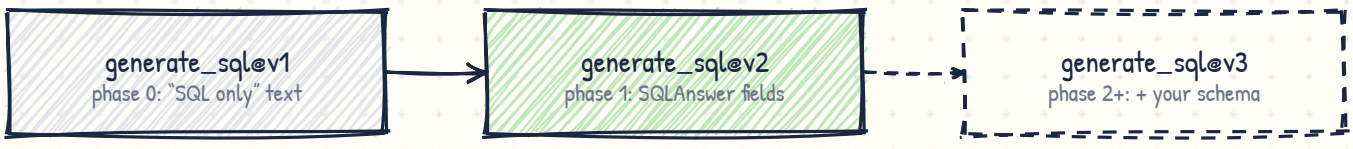


Check	Who	Phase
syntax: does it parse for MySQL / Postgres / …?	sqlglot	1 ✓
statement type: SELECT or DELETE?	sqlglot (AST root)	1 ✓ → guard in 6
semantics: do the tables and columns exist?	schema lookup	2 + 6
result correctness: right rows?	execution accuracy	5

Our badge shows `valid · dialect · statement_types` or the first error with line and column, e.g. *"Line 1, column 37: Expecting)"*.

⚠ **GOTCHA** sqlglot doesn't know your tables: `SELECT x FROM no_such_table` still parses. Invalid SQL is **flagged, never hidden**: the user sees the problem.

Prompt versions



every history row and trace records which version answered → evals can compare them

Interview Q: Why version prompts if they're just strings?

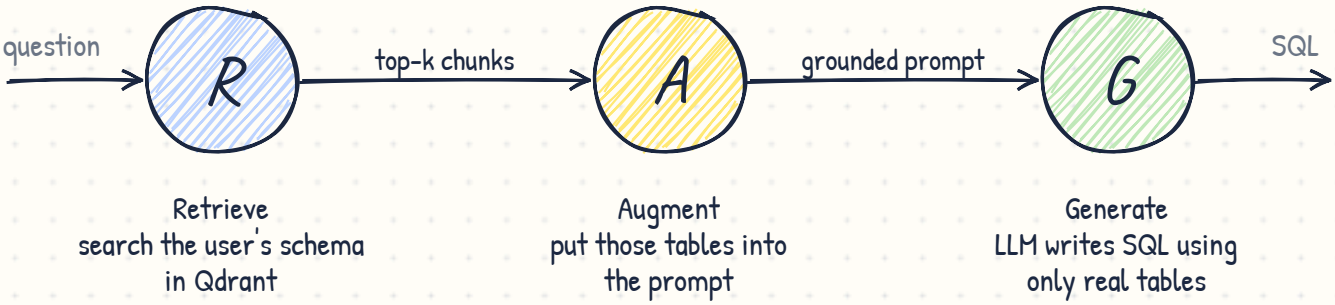
A: A wording change can silently improve one case and break ten others. With the version stored next to every result, Phase 5's evaluation can compare `@v2` and `@v3` on the same questions, and you can roll back.

COMING · phases 2–4

6 RAG

Three ways to give a model new knowledge

Approach	How	Verdict for SQLAssist
Prompt stuffing	paste the whole schema into every prompt	fine for 5 tables; breaks at 500 (context limit, cost, "lost in the middle")
Fine-tuning	retrain weights on examples	teaches style, not facts; stale when the schema changes; one model per customer = impossible
RAG ✓	fetch only the relevant pieces at question time	fresh, per-user, cheap, explainable (we can show which tables were used)



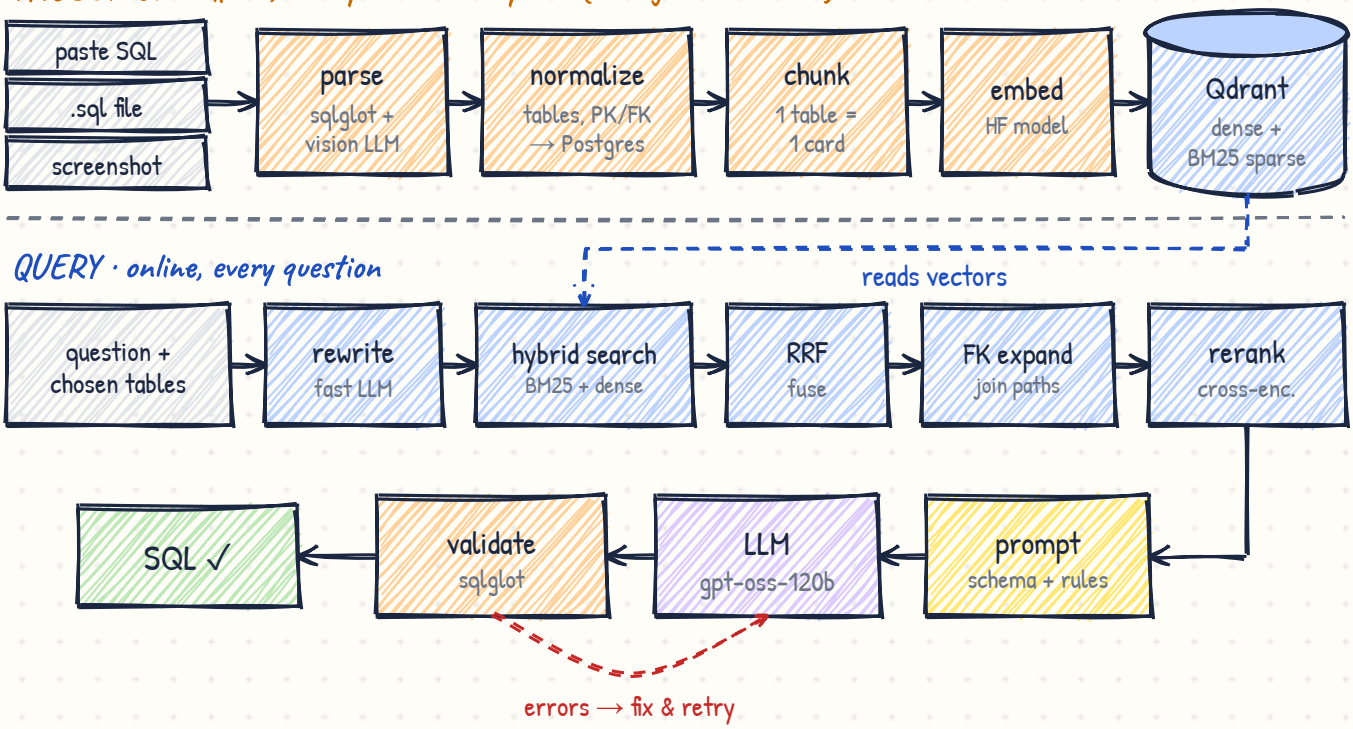
✓ *PRO TIP* "Lost in the middle": models use the start and end of long contexts best. ***Fewer, better chunks*** beat dumping everything.

Interview Q: **RAG or fine-tuning?**

A: RAG for ***knowledge*** that changes or is private (our schemas). Fine-tuning for ***behaviour*** (format, tone, a narrow skill). They combine well.

COMING · phases 2–4

Ingestion vs query pipeline



→ ***Ingestion*** is slow (OCR, embeddings) → it runs as a ***background job***; the UI polls/streams progress.

→ ***Query*** is latency-critical → cheap steps use the fast model; the big model writes the SQL once.

→ Every stage is ***swappable and measurable*** separately; that's what makes evaluation (chapter 7) possible.

✓ PRO TIP **sqlglot** is our secret weapon: it parses SQL in 20+ dialects into an AST (syntax tree). Ingestion uses it to extract tables/columns/keys; validation uses it to check generated SQL.

COMING · phase 2

Chunking

A **chunk** is one retrievable unit of knowledge. Too big → noisy context and a blurry embedding. Too small → meaning gets cut in half.

Strategy	Idea	Good for
fixed-size + overlap	every N tokens, overlapping a bit	plain prose, quick baseline
recursive	split by sections → paragraphs → sentences	documents with structure
semantic	split where the meaning shifts (embedding distance)	long mixed-topic text
structure-aware ✓	split along the data's own units	**schemas (us)**, code, tables

Our unit: one table = one "card"

```
TABLE orders                          (schema: sales)
Columns:
  id              INT          PK
  customer_id     INT          FK → customers.id
  total_amount    DECIMAL(10,2)
  status          VARCHAR(20)  -- placed|shipped|refunded
  created_at      TIMESTAMP
About: one row per customer order.
Synonyms: purchases, sales, transactions
Joins: customers (customer_id), order_items (order_id)
```

Never split a CREATE TABLE in half: a chunk holding half the columns causes exactly the hallucinations we want to stop.

The **About** and **Synonyms** lines are written by the LLM during ingestion, so "purchases" finds `orders`.

Very wide tables (100+ columns) get extra column-group chunks that point back to their parent table.

Payload (metadata) travels with every chunk

```
{ "workspace_id": "w_42", "schema_id": "s_7", "table": "orders",
  "columns": ["id", "customer_id", "total_amount", "status", "created_at"],
  "fk_targets": ["customers", "order_items"], "dialect": "postgres" }
```

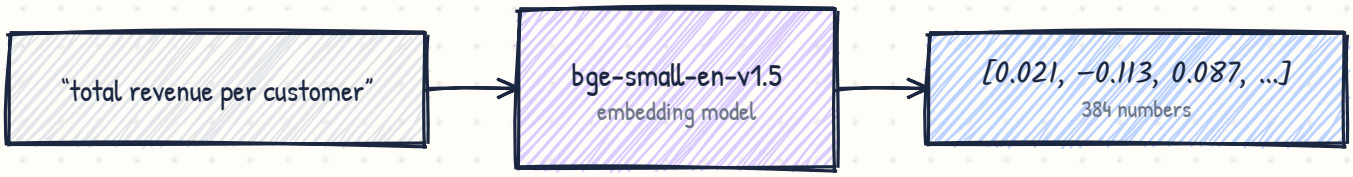
→ **filter**: search only the user's workspace, or only the tables they picked in the UI

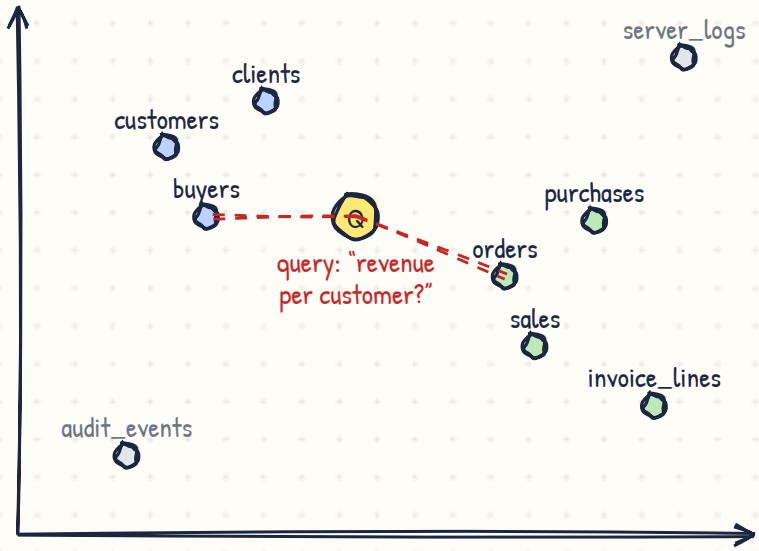
→ **explain**: show the user which tables the answer was built from

→ **expand**: follow `fk_targets` to pull in join tables (FK-graph expansion)

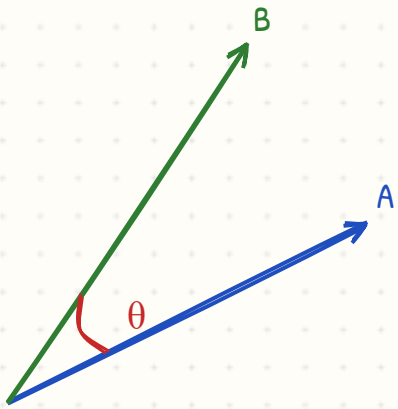
COMING · phase 3

Embeddings: meaning as coordinates





similar meaning ⟶ nearby points (2-D sketch of 384-D space)



$\cos\theta = A \cdot B \,/\, (|A|\,|B|)$

→ ***Cosine similarity*** compares directions: 1 = same meaning, 0 = unrelated, −1 = opposite. Length doesn't matter.

→ ***Bi-encoder***: query and documents are embedded *separately* ⟶ document vectors are computed once at ingestion; search is just math.

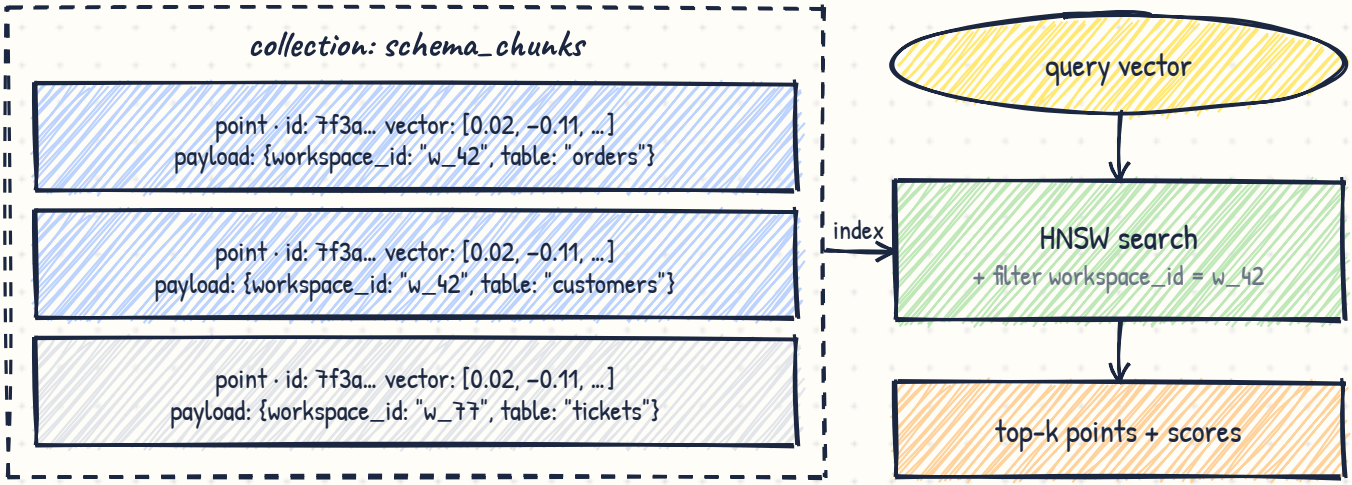
→ Models come from the ***Hugging Face Hub*** (sentence-transformers) and run locally, free.

⚠ GOTCHA Use the ***same model*** for documents and queries. Changing the model = re-embed everything (vectors from different models live in different spaces).

✓ PRO TIP 384 dims (small, fast) vs 768/1024 (richer, more RAM). Start small, measure, upgrade only if recall says so.

COMING · phase 3

Qdrant: a database for vectors

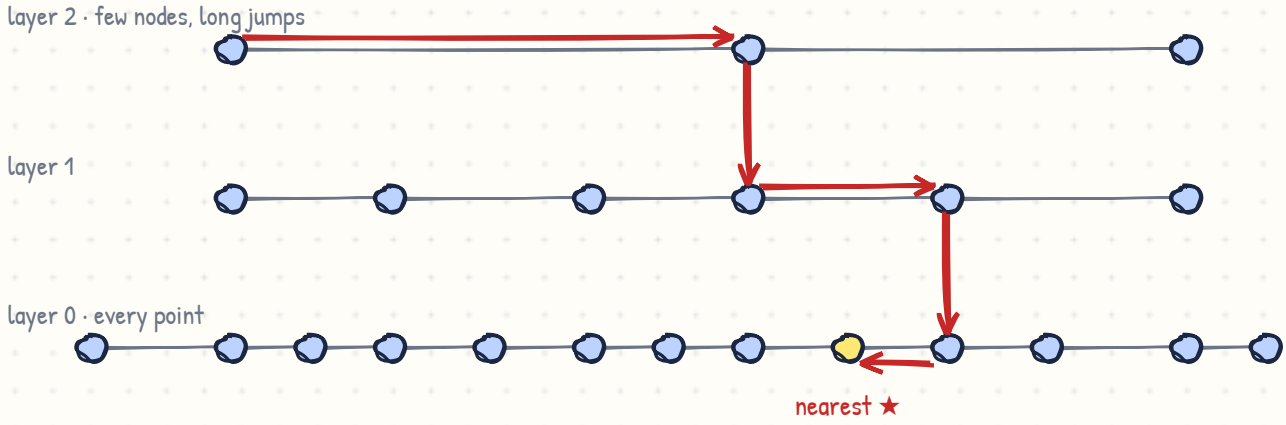


→ ***Point*** = id + vector + ***payload*** (JSON metadata). ***Collection*** = a set of points with the same vector size and distance metric.

→ ***Multitenancy***: one collection for all customers + an indexed `workspace_id` payload field marked as tenant key. Forget the filter = one customer sees another's schema.

→ Qdrant stores ***sparse vectors*** (BM25-style keyword weights) next to dense ones → hybrid search with server-side RRF fusion.

HNSW: how search stays fast



HNSW (Hierarchical Navigable Small World): a layered graph. Start at the sparse top layer, greedily hop toward the query, drop a layer, repeat. It's ***ANN*** (approximate nearest neighbour): it may miss the exact best match rarely, but it's orders of magnitude faster than comparing against every vector.

Interview Q: Why Qdrant instead of FAISS?

A: FAISS is a *library*: an in-process index; you build persistence, filtering, multi-tenancy and an API yourself. Qdrant is a *database server*: REST/gRPC API, payload filters, snapshots, sparse+dense hybrid, horizontal scaling.

COMING · phase 4

Two kinds of search

BM25 · lexical (keywords)

- scores documents by matching ***words***
- ***rare*** words count more (IDF: "cust_ltv" > "id")
- repeating a word helps less and less (saturation, *k1*)
- long documents are normalised (*b*)

✓ exact identifiers: `cust_id`, `tbl_ord_hdr`

✗ blind to synonyms: "buyers" ≠ "customers"

Dense · semantic (embeddings)

- compares ***meaning*** via vectors
- "buyers" ≈ "customers", "income" ≈ "revenue"
- works across phrasing and typos
- needs an embedding model + vector index

✓ paraphrases and vague questions

✗ weak on rare codes and exact names

Hybrid search + Reciprocal Rank Fusion

Run both searches, then merge the *rankings* (not the raw scores, which live on different scales):

$$RRF(d) = \Sigma_{lists}\ 1\ /\ (k + rank(d)) \text{ with } k = 60$$

table	BM25 rank	dense rank	RRF score	final
orders	1	2	1/61 + 1/62 = 0.0325	🥇 1
customers	3	1	1/63 + 1/61 = 0.0323	2
payments	2	—	1/62 = 0.0161	3
buyers_v2	—	3	1/63 = 0.0159	4

→ Documents that ***both*** searches like float to the top: agreement wins.

→ *k* = 60 damps the difference between rank 1 and rank 2, so one list can't dominate.

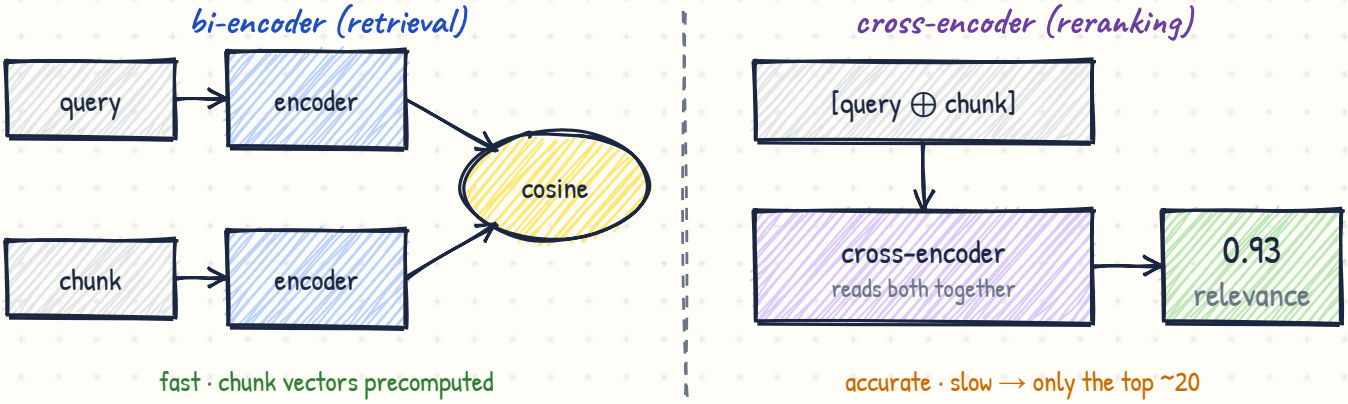
→ No score normalisation needed: only positions are used. That's why RRF is the default fusion in most vector DBs, Qdrant included.

Interview Q: Why not just add the two scores?

A: BM25 scores are unbounded (e.g. 7.3, 12.9) while cosine similarity is in [−1, 1]. Adding them lets one scale dominate. RRF uses ranks only, so it needs no tuning to combine heterogeneous retrievers.

COMING · phase 4

Reranking: a second, smarter opinion



Pattern: ***retrieve wide*** (top 50 with cheap hybrid search) → ***rerank narrow*** (cross-encoder scores 20–50 pairs) → keep the best 5–8 for the prompt. Model: `bge-reranker-base` from Hugging Face.

Fixing the question before searching

Technique	What it does	Example
query rewriting	fast LLM rewrites into schema-friendly terms	"rev last mo" → "monthly revenue, order totals, last month"
multi-query	several phrasings, union of results	"churned users" + "inactive customers" + "cancelled subscriptions"
HyDE	embed a *hypothetical answer* instead of the question	draft SQL/doc → its vector sits close to real matching tables

Our special: foreign-key graph expansion



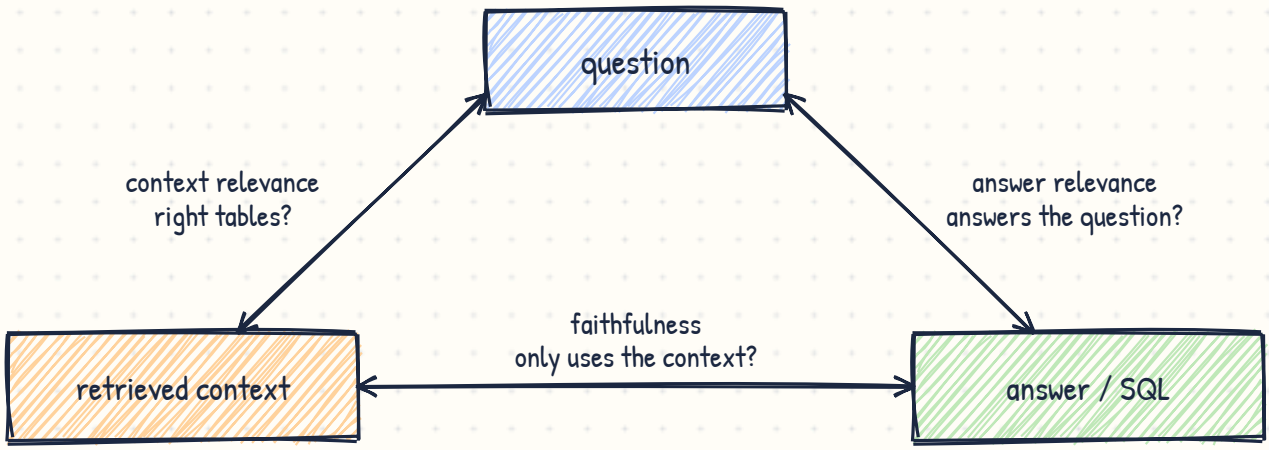
"Revenue per customer *by product*" needs `order_items` even if no search ranked it. We walk the FK graph between retrieved tables to add the missing join path.

COMING · phase 5

7 RAG evaluation

If you can't measure it, you're guessing. Changing the chunk format, embedding model or a prompt can silently make results worse. Evaluation turns "feels better" into numbers you can compare in CI.

The RAG triad



Evaluate the ***retriever*** (did we fetch the right tables?) and the ***generator*** (did we write correct SQL from them?) *separately*, or you can't tell which one broke.

The golden dataset

```
{
  "question": "monthly revenue in 2025",
  "relevant_tables": ["orders"],
  "gold_sql": "SELECT date_trunc('month', created_at) ...",
  "difficulty": "easy",
  "schema": "ecommerce_v1"
}
```

→ ***hand-written***: ~50 high-quality cases you trust
→ ***synthetic***: an LLM drafts questions from the schema; a human reviews them
→ ***benchmarks***: subsets of Spider and BIRD (public text-to-SQL datasets)

	offline eval	*online eval*
when	before deploying (CI, experiments)	in production, on real traffic
data	golden dataset with known answers	user 👍 / 👎, edits to the SQL, failure rates, latency
answers	"is version B better than A?"	"is it working for users right now?"

COMING · phase 5

Retrieval metrics, worked by hand

The question needs 3 tables: **orders**, **customers**, **order_items**. Our retriever returns this top-5:



missed: order_items (relevant but not retrieved)

Metric	Formula	Here	Asks...
Precision@5	relevant retrieved ÷ k	2 / 5 = **0.40**	how much junk did we fetch?
Recall@5	relevant retrieved ÷ all relevant	2 / 3 = **0.67**	did we miss anything?
Hit rate@5	1 if any relevant in top-k	**1**	did we get at least one?
MRR	1 ÷ rank of first relevant (averaged over questions)	1 / 1 = **1.0**	how soon is the first good hit?
nDCG@5	DCG ÷ ideal DCG	**0.70** (below)	are relevant items ranked high?

nDCG step by step (relevance 1 = relevant, 0 = not; position *i* is discounted by $\log_2(i+1)$):

$DCG = 1/\log_2 2 + 0/\log_2 3 + 1/\log_2 4 + 0 + 0 = 1 + 0 + 0.5 = 1.500$
$IDCG = 1/\log_2 2 + 1/\log_2 3 + 1/\log_2 4 = 1 + 0.631 + 0.5 = 2.131$ *(perfect order: 3 relevant first)*
$nDCG = 1.500 / 2.131 = 0.70$

✓ **PRO TIP** For text-to-SQL, **recall** is king: one missing table = a broken JOIN. We'd rather send 6 tables with 1 extra than 4 with 1 missing. Rerankers then fix precision.

Interview Q: MRR vs nDCG?

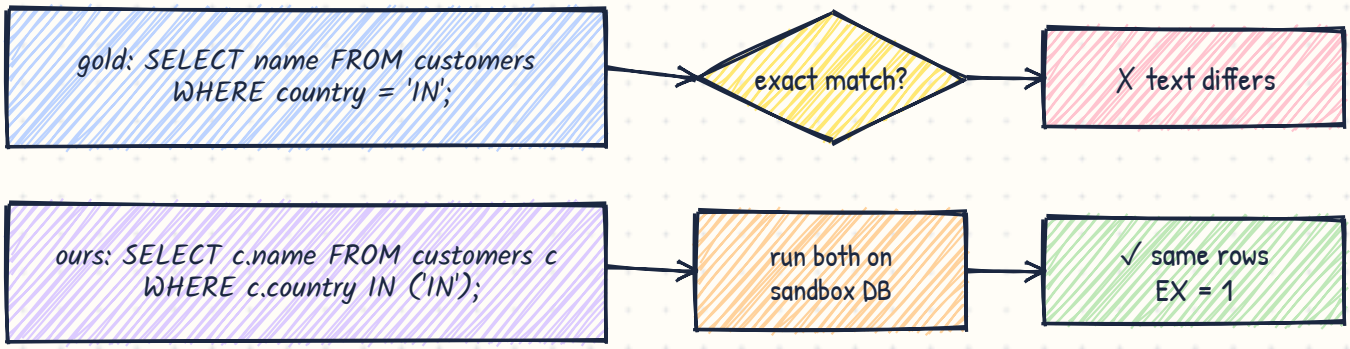
A: MRR only looks at the *first* relevant hit (good when one answer suffices). nDCG rewards the *whole* ranking, with graded relevance; better when several items are needed, like our tables.

COMING · phase 5

Judging the generated answer

Metric (RAGAS-style)	Question it answers	Needs ground truth?
faithfulness	is every claim in the answer supported by the retrieved context? (anti-hallucination)	no
answer relevancy	does the answer address what was asked?	no
context precision	are the useful chunks ranked at the top?	yes
context recall	does the context contain everything the gold answer needs?	yes

Text-to-SQL: exact match vs execution accuracy

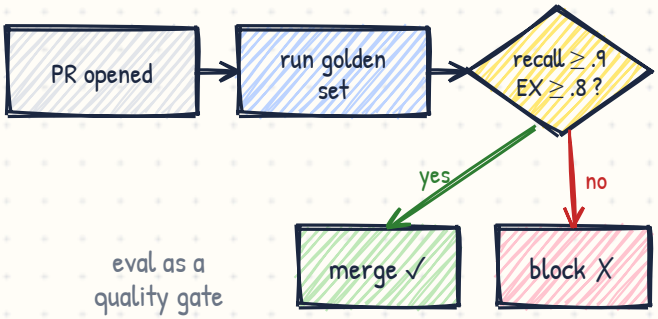


→ ***Valid-SQL rate***: does it parse for the dialect (sqlglot)? Cheapest check, run on everything.

→ ***Exact match (EM)***: same text/AST as gold. Too strict: many correct queries look different.

→ ***Execution accuracy (EX)***: run predicted and gold SQL on a seeded sandbox database and compare result sets. The gold standard (used by Spider & BIRD).

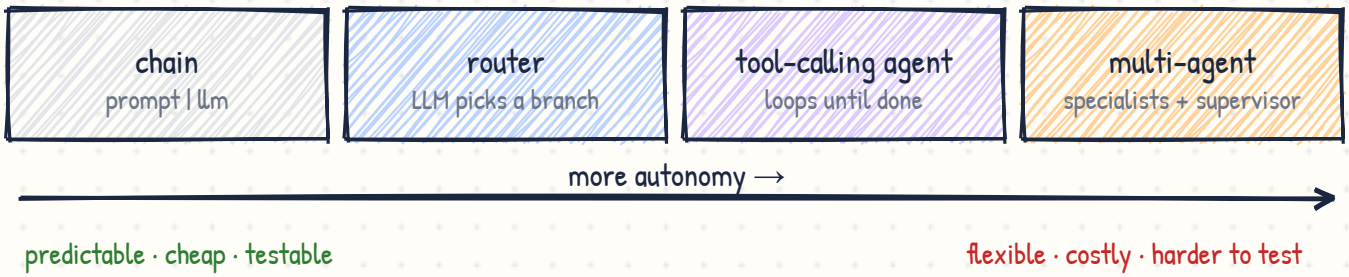
LLM-as-judge: a strong model grades answers against a rubric. Known biases: *position* (prefers the first option), *verbosity* (prefers longer), *self-preference* (likes its own family's outputs). Mitigate: clear rubrics, swap order, spot-check with humans.



COMING · phase 6

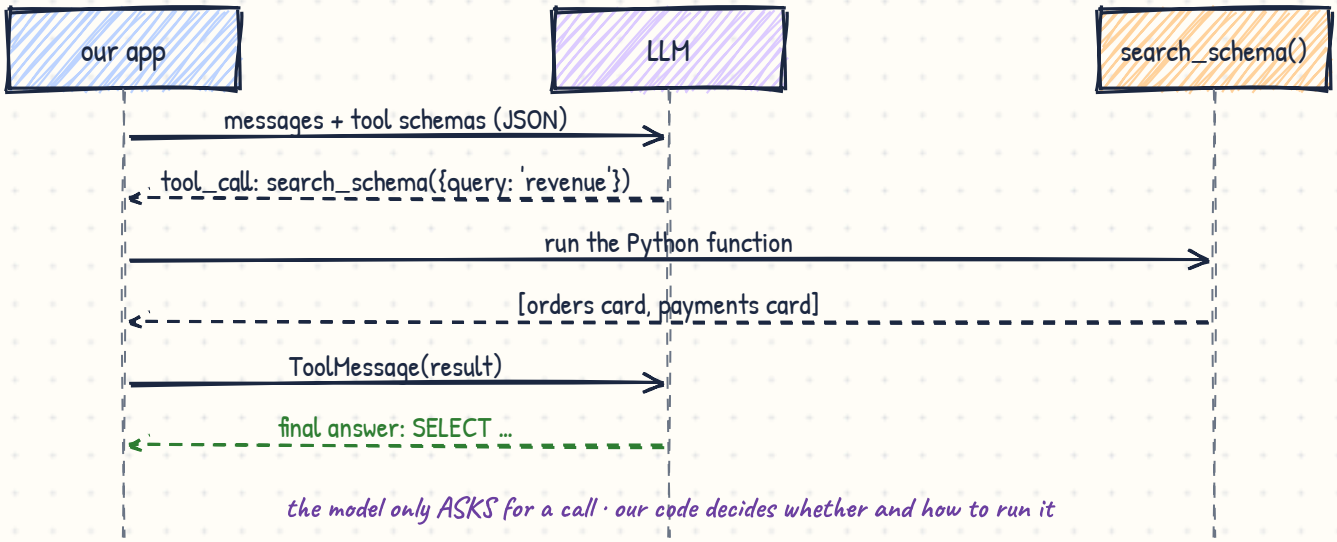
8 Agents

An **agent** is an LLM that decides what to do next : it calls tools, looks at the results and loops until the task is done. **Agent = LLM (brain) + tools (hands) + state/memory + a control loop.**



✓ PRO TIP Use the **least autonomy** that solves the problem. Our SQL flow is mostly a fixed graph with a few LLM decisions, not a free-roaming agent.

Tool calling, step by step

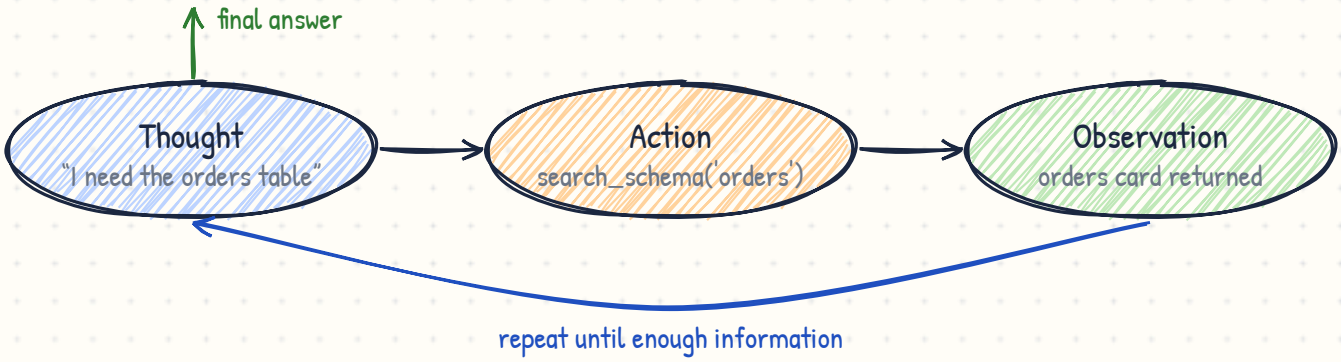


Interview Q: Can the LLM run code or query the database by itself?

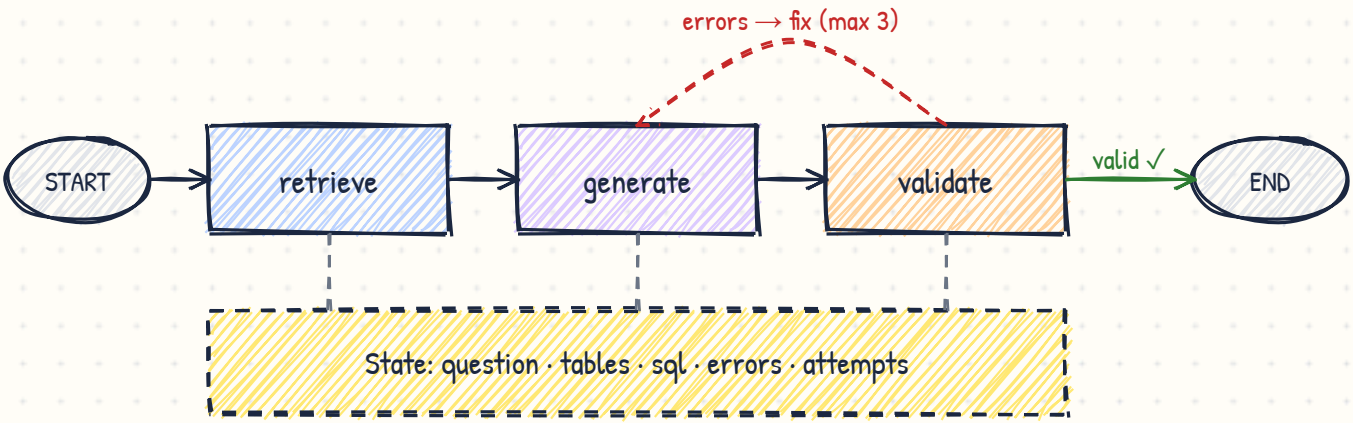
A: No. It outputs a structured request (tool name + JSON arguments). Our application validates it, executes it with *our* permissions, and returns the result as a message. That boundary is where guardrails live.

COMING · phase 6

ReAct: reason + act



LangGraph: agents as state machines

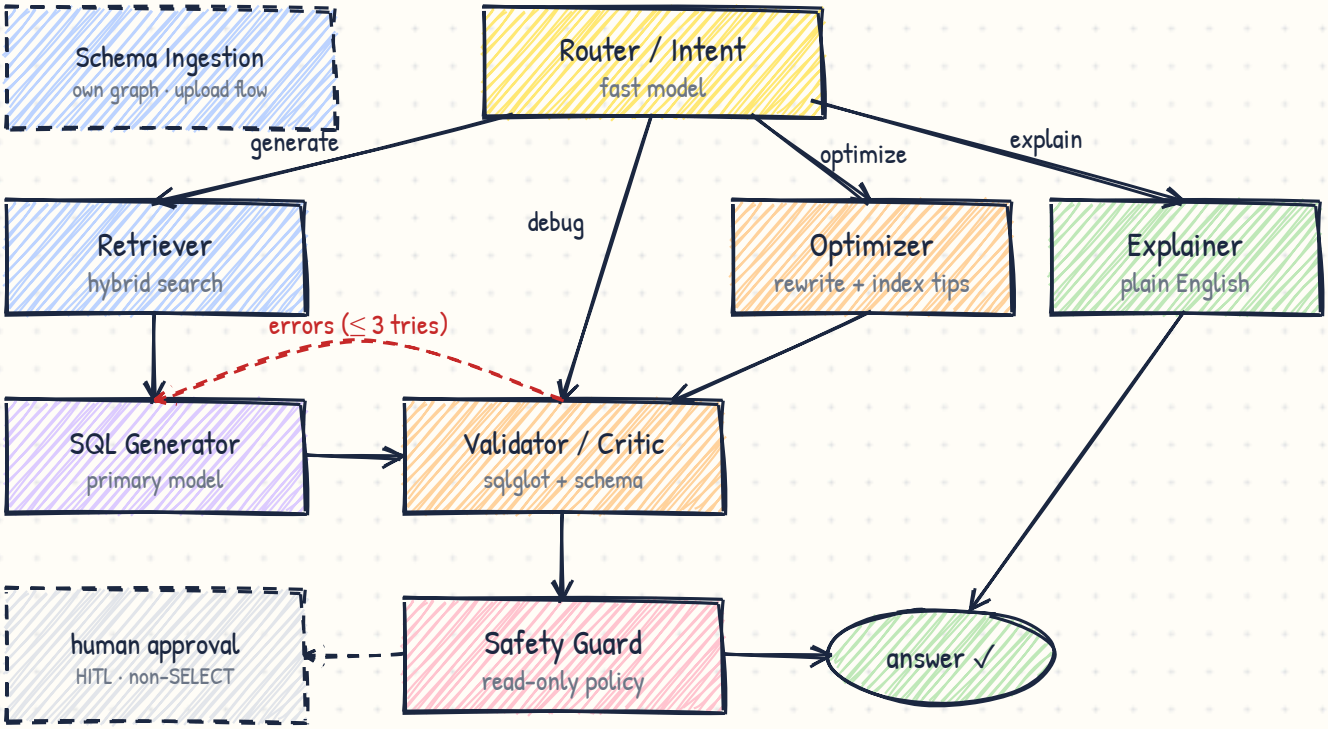


→ ***State***: a typed dict shared by all nodes.
→ ***Node***: a function `state → partial update` (may call an LLM or a tool).
→ ***Edge***: fixed next step. ***Conditional edge***: a function picks the next node.
→ ***Reducer***: how updates merge (e.g. `add_messages` appends instead of overwriting).
→ ***Checkpointer***: saves state after every step → memory, resume, human-in-the-loop.

```
graph = StateGraph(SQLState)
graph.add_node("retrieve", retrieve)
graph.add_node("generate", generate)
graph.add_node("validate", validate)
graph.add_edge(START, "retrieve")
graph.add_edge("retrieve", "generate")
graph.add_edge("generate", "validate")
graph.add_conditional_edges("validate",
    route, {"fix": "generate", "done": END})
app = graph.compile(checkpointer=saver)
```

COMING · phase 6

The SQLAssist agent team



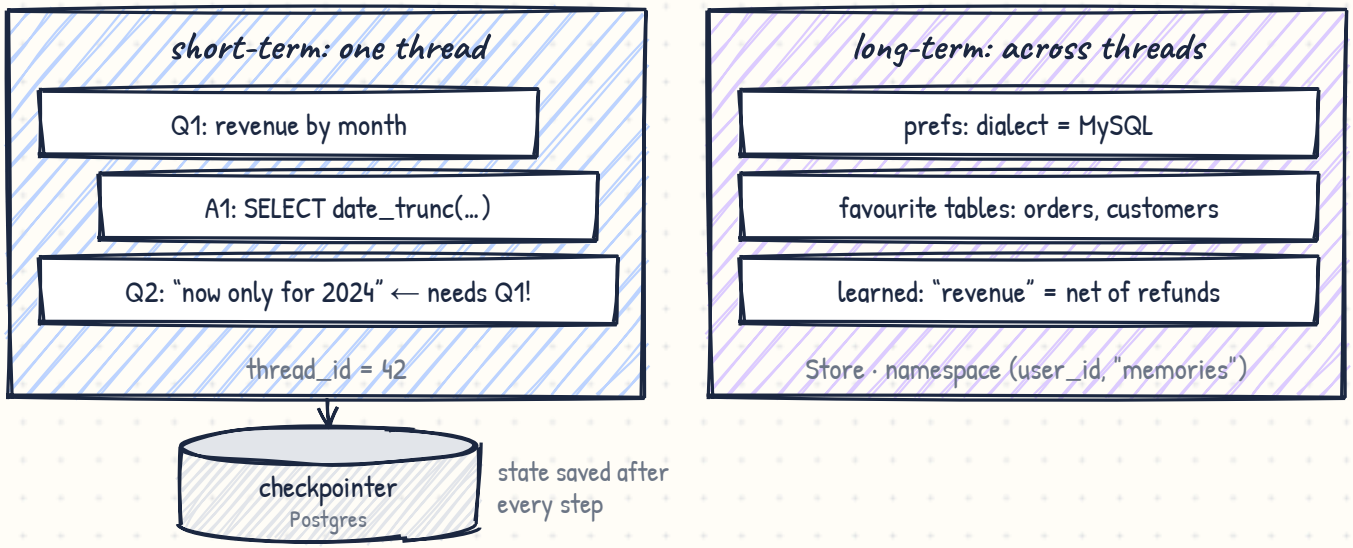
dashed red = reflection loop: the critic's errors are fed back to the generator

Agent	Job	Model	Tools
Router	classify intent: generate / debug / optimize / explain / chat	fast	—
Schema Ingestion	turn messy input (text, screenshots) into DDL; ask when unclear	primary + vision	sqlglot
Retriever	rewrite query, hybrid search, retry if confidence is low	fast	Qdrant, FK graph
SQL Generator	write SQL from retrieved tables only	primary	—
Validator / Critic	parse, check every table/column exists, report errors	fast	sqlglot, schema lookup
Safety Guard	block destructive SQL & prompt injection	fast + Prompt Guard	AST policy
Optimizer · Explainer	performance rewrite + index tips · plain-English walkthrough	primary · fast	sqlglot

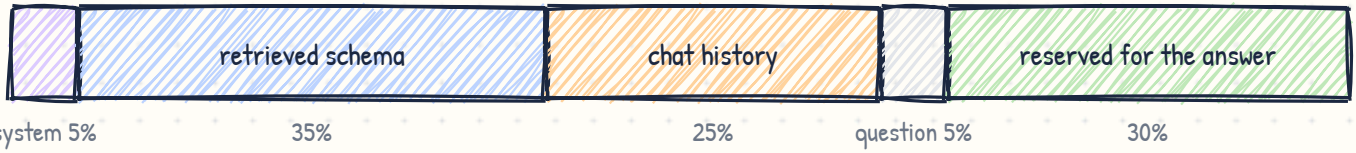
COMING · phase 6

9 Memory management

LLMs are ***stateless***: every call starts from zero. "Memory" is whatever *we* decide to put back into the context window.



The context budget (it all has to fit)



→ ***trim***: keep only the last N turns
→ ***summarise***: fast model compresses old turns into a paragraph
→ ***retrieve***: store past turns as vectors, fetch only relevant ones (memory as RAG)
→ ***drop tool outputs*** once used (big schema cards add up fast)

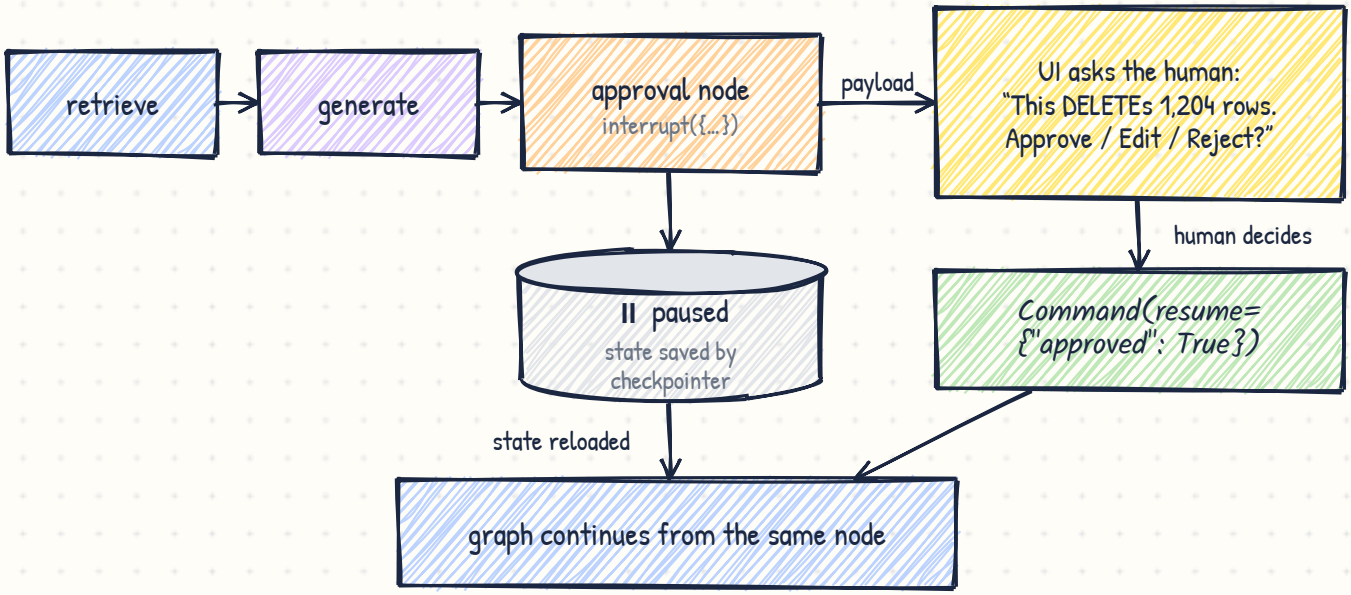
Memory types (LangGraph vocabulary): *semantic* = facts ("user prefers MySQL"), *episodic* = past experiences/examples (a query the user corrected), *procedural* = instructions (prompt rules the agent refines).

⚠ GOTCHA Memory is ***user data***: tenant-isolated, and deleted with the account.

COMING · phase 6

10 Human-in-the-loop

HITL = the graph pauses at a chosen point, a human answers / approves / edits, and the graph resumes exactly where it stopped.



Patterns

→ **approve / reject** a risky action
→ **edit state**: fix tables or SQL before continuing
→ **answer a clarification**: "gross or net revenue?"
→ **review tool calls** before they run

Where SQLAssist uses it

→ ambiguous question → ask instead of guessing
→ confirm selected tables before generating
→ any non-SELECT statement needs explicit approval

```python
from langgraph.types import interrupt, Command

def approval(state):
    decision = interrupt({"sql": state["sql"]})
    return {"approved": decision["approved"]}

# later, when the user clicks Approve:
graph.invoke(Command(resume={"approved": True}),
             config={"configurable": {"thread_id": "42"}})
```

⚠ **GOTCHA** HITL **requires a checkpointer** (Postgres-backed in production) so a paused run survives restarts, and the same `thread_id` to resume the right run.

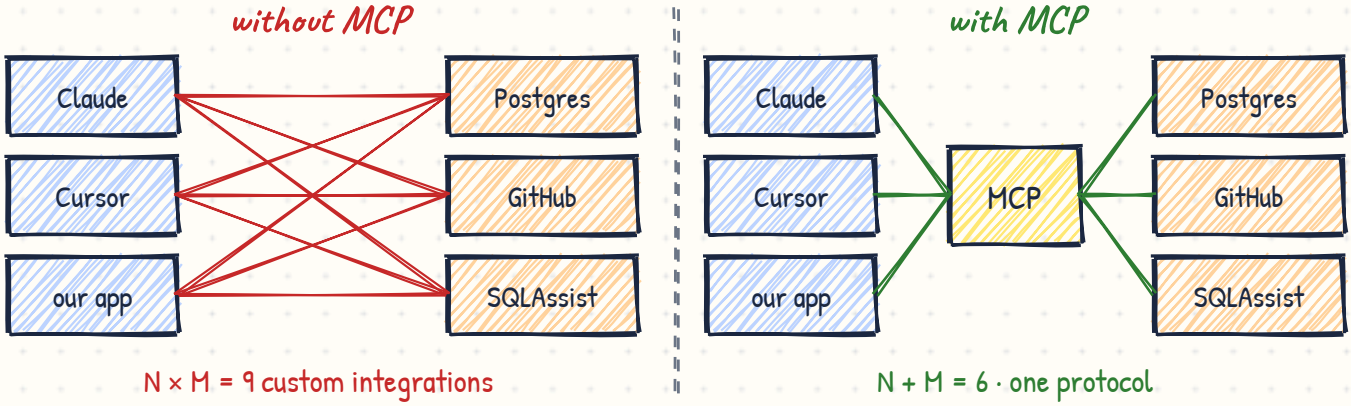
Interview Q: Why HITL instead of a better prompt?

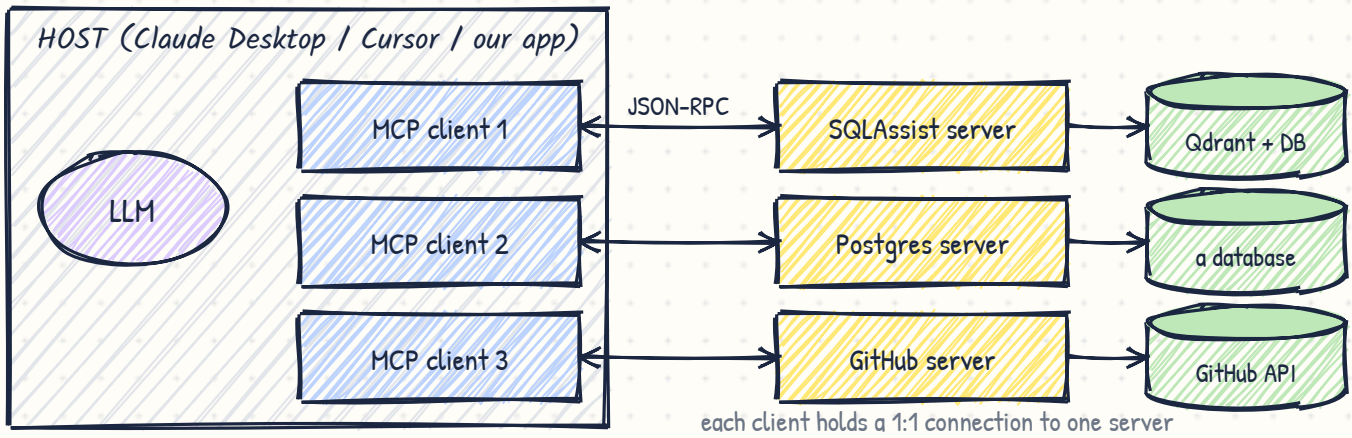
A: Some decisions need information or *authority* only the human has (which revenue definition, whether deleting is OK). Prompts can't create that; a pause can. It also builds trust and audit trails.

COMING · phase 7

11 MCP · Model Context Protocol

An open standard for connecting AI applications to tools and data: "USB-C for AI apps". Build a tool once as an MCP ***server***; any MCP-capable app can use it.





Primitive	Offered by	Controlled by	Example in SQLAssist
Tools	server	the model (it decides to call)	`generate_sql` , `validate_sql`
Resources	server	the application (attaches as context)	`schema://w_42/orders` table card
Prompts	server	the user (picks a template)	"optimize this query" template
Sampling · Roots · Elicitation	client	host/user	server borrows the host's LLM · allowed paths · asks user for input

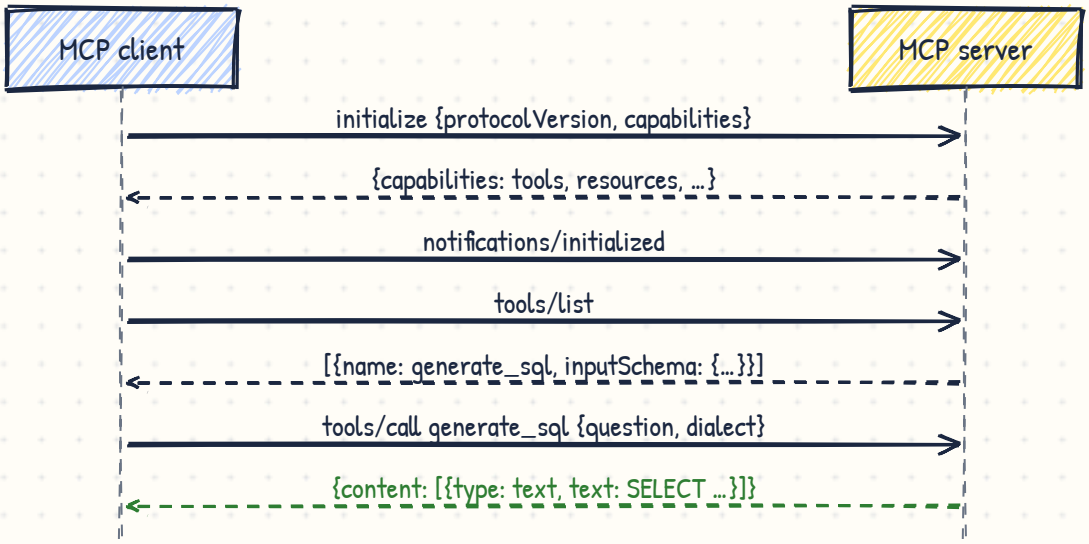
COMING · phase 7

Under the hood: JSON-RPC 2.0

```
// client → server
{ "jsonrpc": "2.0", "id": 7,
  "method": "tools/call",
  "params": {
    "name": "generate_sql",
    "arguments": {
      "question": "monthly revenue",
      "dialect": "PostgreSQL" } } }
```

```
// server → client (same id)
{ "jsonrpc": "2.0", "id": 7,
  "result": {
    "content": [
      { "type": "text",
        "text": "SELECT date_trunc(..." }
    ] } }
```

Lifecycle



Transport	How	Use when
stdio	host starts the server as a subprocess; messages over stdin/stdout	local tools on the same machine (Claude Desktop config)
Streamable HTTP	HTTP POST per message, optional SSE streaming back; OAuth for auth	remote / multi-user servers (our SaaS)

✓ PRO TIP **Our plan (phase 7):** a FastMCP server exposing `search_schema`, `generate_sql`, `validate_sql`, `explain_sql`, all calling the same `services/` layer the REST API uses. That's the payoff of chapter 2's layering.

⚠ GOTCHA **Tool output is untrusted.** Text returned by a server can contain prompt-injection ("ignore your instructions…"). Least privilege, per-user auth, and treat results as data.

COMING · phases 8–9

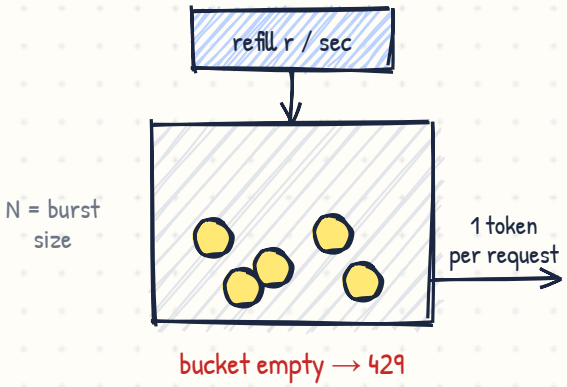
12 Guardrails & scale

Defense in depth



→ ***Prompt injection***: text that tries to override instructions. It can arrive in the question *or in retrieved content* (a malicious schema comment!) → treat retrieved text as data. Groq hosts ***Llama Prompt Guard 2*** classifiers for this.

→ ***Read-only policy***: parse generated SQL with sqlglot and block `DROP / DELETE / UPDATE / ALTER` unless a human approves (chapter 10).



Rate limiting (token bucket): each user's bucket holds N tokens and refills at *r*/second; each request takes one; empty → ***429 Too Many Requests***. Protects us *and* our Groq free-tier quota. Plans get different buckets.

Caching: *exact* cache (same prompt hash → stored answer) and *semantic* cache (a new question whose embedding is ~0.95 similar to a past one reuses the answer). Key it by workspace + schema version or users see stale or foreign answers.

COMING · phases 8–9

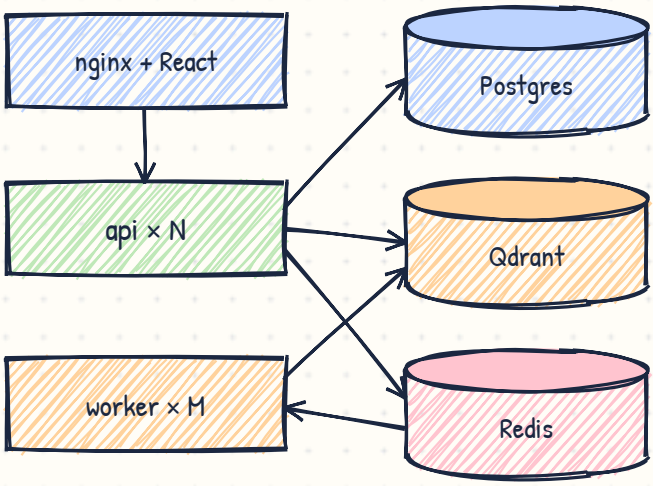
Docker in one minute

→ ***Image*** = recipe + ingredients, frozen (read-only layers: OS, Python, deps, our code).

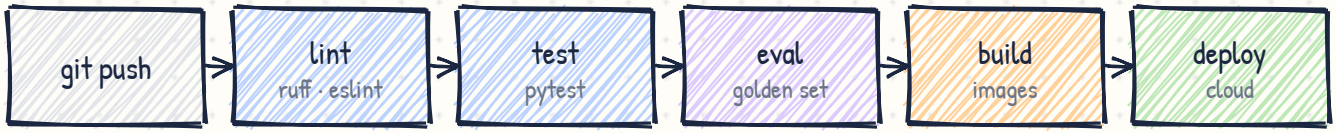
→ ***Container*** = a running instance of an image (the cake). Many containers from one image.

→ ***Multi-stage build***: a big "build" stage installs everything; a slim "runtime" stage copies only what's needed → small, safer images.

→ ***Compose***: one YAML file describes all services; `docker compose up` starts the whole system. (Phase 0 already uses it for Postgres + Qdrant.)



CI/CD pipeline



Scaling & observability

→ ***Stateless API***: JWT auth, no in-memory sessions → run N replicas behind a load balancer.

→ ***State lives in services***: Postgres, Qdrant, Redis (each scales on its own).

→ ***Slow work → queue + workers*** (ingestion, evals), scaled separately from the API.

→ ***LLM limits***: queue + backoff + cache + model tiers (fast model for cheap steps).

Signal	Answers
logs	what happened?
metrics	how many, how fast? (p95 latency, tokens, cost)
traces	which path did *this* request take through agents and tools?

LLM-specific tracing (prompts, tokens, each agent step): ***Langfuse*** (self-hostable) or LangSmith.

✓ **PRO TIP** Already built: `/api/v1/health/live` (is the process up? → orchestrator restarts it if not) and `/api/v1/health/ready` (can it reach the DB? → load balancer stops sending traffic if not).

Glossary · A–L

Agent: LLM + tools + state + loop; decides its next step itself.

Alembic: migration tool for SQLAlchemy; versioned schema changes.

ANN: approximate nearest neighbour search; trades a little accuracy for big speed.

ASGI: async interface between Python web servers (Uvicorn) and apps (FastAPI).

AST: abstract syntax tree; code parsed into typed nodes (sqlglot builds one per SQL statement).

async / await: cooperative concurrency; `await` yields while waiting on I/O.

Backoff (exponential): wait longer after each failed retry (0.5 s, 1 s, 2 s…).

bcrypt: slow, salted password hash with a tunable cost factor.

Bi-encoder: embeds query and document separately; fast retrieval.

BM25: classic keyword ranking function (term frequency, IDF, length normalisation).

Checkpointer: LangGraph component that saves graph state after each step.

Chunk: one retrievable unit of knowledge (for us: a table card).

Conditional edge: graph edge whose target is chosen by a function at runtime.

Constrained decoding: the decoder may only sample tokens that keep the output valid for a schema.

Context window: max tokens a model can attend to in one call (input + output).

CORS: browser rule; servers must allow other origins to read their responses.

Cosine similarity: angle-based similarity of two vectors, −1 … 1.

Cross-encoder: reads query + document together to score relevance; used for reranking.

Dependency injection: a function declares needs; the framework provides them.

Embedding: vector of numbers representing meaning; similar meaning → close vectors.

Event loop: single thread that runs async tasks, switching at `await` points.

Execution accuracy (EX): predicted SQL returns the same rows as the gold SQL.

Faithfulness: answer is fully supported by the retrieved context.

Fallback: switch to an alternative (e.g. another model) when the first choice fails.

FastMCP: Python framework for building MCP servers with decorators.

Fine-tuning: further training a model's weights on your examples.

Golden dataset: hand-checked questions with known correct answers for evaluation.

Groq: inference provider running open LLMs on custom LPU hardware.

Guardrail: check that blocks or fixes unsafe inputs/outputs around an LLM.

Hallucination: fluent but false model output.

HITL: human-in-the-loop; the graph pauses for a human decision, then resumes.

HNSW: layered proximity graph used for fast ANN vector search.

Hybrid search: keyword (BM25) + semantic (dense) search, fused (e.g. RRF).

HyDE: embed a hypothetical answer instead of the question to improve retrieval.

IDOR: insecure direct object reference; accessing others' data by changing an id.

Jitter: randomness added to retry delays so clients don't retry in sync.

Idempotent: repeating a request has the same effect as doing it once (PUT, DELETE).

JSON-RPC: lightweight request/response protocol; the wire format of MCP.

JSONB: PostgreSQL's binary JSON type: queryable and indexable.

JWT: signed token (header.payload.signature) carrying claims like user id + expiry.

LangChain: framework of composable LLM components (prompts, models, parsers, tools).

LangGraph: library for stateful agent workflows as graphs, with memory and HITL.

LCEL: LangChain Expression Language; `a | b | c` composes Runnables.

LLM-as-judge: using a strong model with a rubric to grade outputs.

LPU: Groq's Language Processing Unit chip, built for fast inference.

Glossary · M–Z

MCP: Model Context Protocol; open standard connecting AI hosts to tool/data servers.

Middleware: code wrapping every request/response (CORS, logging, rate limits).

Migration: versioned script that changes the database schema (up and down).

MoE: mixture-of-experts; a router activates only a few expert sub-networks per token.

MRR: mean reciprocal rank; average of 1 / rank of the first relevant result.

nDCG: ranking quality metric that rewards relevant items placed higher.

ORM: object-relational mapper; Python classes ⇄ database tables.

Payload: JSON metadata stored with a vector in Qdrant; used for filtering.

Precision@k: share of the top-k results that are relevant.

Preflight: browser's OPTIONS request asking the server if a cross-origin call is allowed.

Prompt injection: input that tries to override the model's instructions.

Prompt template: reusable prompt with placeholders filled at runtime.

Pydantic: data validation via Python type hints; powers FastAPI request/response models.

Qdrant: open-source vector database (dense + sparse vectors, payload filters).

RAG: retrieval-augmented generation; retrieve context, then generate with it.

Rate limiting: capping requests per user/time window (e.g. token bucket) → 429.

ReAct: agent pattern alternating Thought → Action → Observation.

Reasoning tokens: hidden "thinking" a reasoning model generates before answering; billed as output.

Recall@k: share of all relevant items that appear in the top-k.

Reducer: rule for merging a node's update into LangGraph state.

Reflection: a critic's feedback is fed back so the generator can fix its output.

Refresh token: long-lived token used only to obtain new access tokens.

Reranker: model that re-scores retrieved candidates more precisely (cross-encoder).

Retry-After: HTTP header telling a client how long to wait (sent with 429).

RRF: reciprocal rank fusion; merge rankings with Σ 1/(k + rank).

Runnable: LangChain interface with invoke / ainvoke / stream / batch.

Salt: random per-password value mixed into a hash to defeat precomputed tables.

Semantic cache: reuse answers for questions whose embeddings are near-identical.

SSE: Server-Sent Events; one HTTP response the server keeps writing events into.

Session (DB): SQLAlchemy unit of work; tracks objects and commits transactions.

sqlglot: SQL parser/transpiler for 20+ dialects; produces an AST.

Sparse vector: mostly-zero vector of keyword weights (BM25-style) for lexical search.

State (LangGraph): typed dict shared by all graph nodes.

Structured output: the model returns data matching a schema instead of free text.

Stateless API: server keeps no per-user memory between requests; easy to replicate.

Streamable HTTP: MCP's remote transport (HTTP POST + optional SSE stream).

Temperature: sampling knob; low = focused/deterministic, high = random/creative.

Token: unit of text an LLM reads and writes (≈ 4 English characters).

Trace / span: the tree of timed steps one request went through / one step in it.

Tool calling: model outputs a structured function call that the app executes.

Transaction: group of DB writes that succeed or fail together (ACID).

TTFT: time to first token; the wait before the first visible output.

UUID: 128-bit random identifier; non-guessable primary keys.

Uvicorn: ASGI web server that runs our FastAPI app.

Vector database: database optimised for storing and searching embeddings.

DONE ✓ 2026-10-07

0 Phase 0 · Foundation

What we built

→ Backend rebuilt in layers: *core/ db/ models/ schemas/ services/ llm/ api/*
→ MongoDB → **PostgreSQL** (SQLAlchemy 2 async + asyncpg) with **Alembic** migrations
→ **bcrypt** passwords, **JWT access + refresh**, protected routes, */users/me*
→ **LangChain** LCEL chains on Groq **gpt-oss**, per-mode prompts, model tiers in config
→ History saved server-side, paginated, owner-scoped
→ One error envelope + real status codes · health checks
→ Frontend: *lib/api.js* client, auto token refresh, *ProtectedRoute*
→ uv + pyproject, docker-compose (Postgres + Qdrant), this notebook

How we verified it

→ 23 pytest tests (SQLite in memory + fake LLM)
→ live Groq call through the real chain (~1.1 s)
→ migration up / down / drift check
→ browser E2E: signup → generate → history → silent token refresh

Bugs fixed

Bug	Lesson
deprecated Groq model → every AI call failed	model names in config
history never saved (un-awaited async calls)	async needs *await*
plain-text passwords	hash with bcrypt
no auth on any route; IDOR on /user-data	identity from token only
errors returned as HTTP 200	status codes matter
sync LLM client blocked the event loop	use *ainvoke*
login stored "undefined" token, still navigated	check *response.ok*
signup expected 200; no confirm field	201 Created
toString(Date.now()) ids	global vs method
profile fetch wiped plan/bio fields	merge, don't replace
"explain" forced SQL-only prompt	one prompt per task
13-char JWT secret	≥ 256-bit secrets
empty requirements.txt, 2 package.json	reproducible installs
hard-coded localhost URLs	config via env vars

My takeaways: fix the foundation before adding AI · every boundary (HTTP, DB, LLM) needs explicit error handling · test with fakes, verify with the real thing · the LLM never saw the schema, which is exactly why phases 2–4 (RAG) exist.

Next → Phase 1 (LLM layer): structured JSON outputs, streaming tokens to the UI, token counting & cost tracking, prompt versioning, first LangSmith/Langfuse traces.

DONE ✓ 2026-10-07

1 Phase 1 · LLM layer

What we built

→ ***Structured output***: `SQLAnswer` via strict JSON-schema decoding (explanation, assumptions, tables)
→ ***sqlglot validation*** badge on every query: dialect, statement types, errors with line + column
→ ***SSE streaming*** for explanations (FastAPI `EventSourceResponse` → `fetch` stream reader)
→ ***Tokens, latency, cost*** per call, incl. reasoning + cached tokens; migration 2 (7 columns, JSONB details)
→ ***Reasoning effort*** switch (Auto / Fast / Balanced / Thorough) with per-prompt defaults
→ ***Retry vs fallback*** policy; 429 + `Retry-After`; Groq SDK's hidden retries switched off
→ ***Prompt versions*** (`generate_sql@v2`) in history and traces
→ ***LangSmith tracing*** (opt-in) · DB connection released during LLM calls

How we verified it

→ 51 pytest tests with a fake provider (structured, streaming, 429s, outages)
→ migration up / down / drift check on your real PostgreSQL
→ live API run: generate, debug, streamed explain, history with tokens + cost
→ browser: Examples, effort switch, badges, streaming sampled every 100 ms

Measured, then changed

explain (20b)	medium	low ✓
reasoning tokens	1,129	6
first visible word	1.72 s	0.69 s
whole answer	2.27 s	1.03 s
cost	$0.00049	$0.00011

Problems found & fixed

Problem	Lesson
SDK slept up to 60 s per 429, twice	read your SDK's defaults
DB connection held during LLM calls	release before slow I/O
"Examples" crashed the page (since commit 1)	lint catches real bugs
blank API key in .env = "" not None	validate config edges
a test leaked env vars to other tests	inject, don't mutate globals

My takeaways: structure what code consumes, stream what people read · a schema with constrained decoding is a guarantee, a prompt is a wish · measure before you tune (reasoning effort) · retry the transient, fall back on the overloaded · every result carries its prompt version, so it can be evaluated later.

Next → Phase 2 (schema ingestion): paste SQL, upload a `.sql` file or a screenshot → sqlglot + a vision model extract tables, columns and keys → stored in PostgreSQL → the Database Setup page shows *your* schema, and prompts finally get real table names.